

# 시간 민감도에 따른 항공사 추천

1조 - 이게 최선이조

엄창현, 이도윤, 이수아, 최홍준

# 목차

|    |                 |     |
|----|-----------------|-----|
| 01 | 분석 개요           | 03p |
| 02 | 데이터 전처리, EDA    | 04p |
| 03 | 모델 설계 및 데이터셋 예측 | 32p |
| 04 | 결론 도출           | 53p |



# 분석 개요

시간, 가격 데이터를 통해  
개인의 우선도에 맞춘  
항공 노선을 추천해주는  
회귀 모델 구축

## Columns

| 컬럼               | 설명                         |
|------------------|----------------------------|
| index            | 데이터 인덱스                    |
| airline          | 항공사 (6개)                   |
| flight           | 항공편 식별자                    |
| source_city      | 출발 도시 (6개)                 |
| destination_city | 도착 도시 (6개)                 |
| departure_time   | 출발 시간대                     |
| arrival_time     | 도착 시간대                     |
| duration         | 비행 소요 시간 (시간 단위)           |
| price            | 항공권 가격 (₹ 기준)              |
| class            | 좌석 등급 (Economy / Business) |
| stops            | 경유 횟수 또는 직항 여부             |
| days_left        | 출발일까지 남은 예약 일수             |

# 데이터 전처리, EDA

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import re
```

```
print(df.isnull().sum())
```

```
---<데이터 결측치 확인 (isnull)>---
```

```
index          0
airline         0
flight          0
source_city     0
departure_time  0
stops           0
arrival_time    0
destination_city 0
class           0
duration        0
days_left      0
price           0
dtype: int64
```

```
print("---<중복 행 확인>---")
print("중복 행 수:", df.duplicated().sum())
```

```
---<중복 행 확인>---
중복 행 수: 0
```

## # 'flight' 컬럼, 'index' 컬럼 제거

```
df_cleaned=df.drop(columns=['index','flight'])  
print("---<불필요한 컬럼 제거 후>---")  
print(df_cleaned.head())
```

---<불필요한 컬럼 제거 후>---

|   | airline  | source_city | departure_time | stops | arrival_time  | destination_city | ₩ |
|---|----------|-------------|----------------|-------|---------------|------------------|---|
| 0 | SpiceJet | Delhi       | Evening        | zero  | Night         | Mumbai           |   |
| 1 | SpiceJet | Delhi       | Early_Morning  | zero  | Morning       | Mumbai           |   |
| 2 | AirAsia  | Delhi       | Early_Morning  | zero  | Early_Morning | Mumbai           |   |
| 3 | Vistara  | Delhi       | Morning        | zero  | Afternoon     | Mumbai           |   |
| 4 | Vistara  | Delhi       | Morning        | zero  | Morning       | Mumbai           |   |

|   | class   | duration | days_left | price |
|---|---------|----------|-----------|-------|
| 0 | Economy | 2.17     | 1         | 5953  |
| 1 | Economy | 2.33     | 1         | 5953  |
| 2 | Economy | 2.17     | 1         | 5956  |
| 3 | Economy | 2.25     | 1         | 5955  |
| 4 | Economy | 2.33     | 1         | 5955  |

## # 'stops' 컬럼 값을 숫자로 변경

```
df_cleaned['stops']=df_cleaned['stops'].replace({'zero': 0, 'one': 1, 'two_or_more': 2})
```

```
print("---<'stops' 컬럼 숫자 변환 후>---")
```

```
print(df_cleaned.head())
```

```
print("---<'stops' 컬럼 숫자 변환 후>---")
```

```
print(df_cleaned.head())
```

---<'stops' 컬럼 숫자 변환 후>---

|   | airline  | source_city | departure_time | stops | arrival_time  | destination_city | ₩ |
|---|----------|-------------|----------------|-------|---------------|------------------|---|
| 0 | SpiceJet | Delhi       | Evening        | 0     | Night         | Mumbai           |   |
| 1 | SpiceJet | Delhi       | Early_Morning  | 0     | Morning       | Mumbai           |   |
| 2 | AirAsia  | Delhi       | Early_Morning  | 0     | Early_Morning | Mumbai           |   |
| 3 | Vistara  | Delhi       | Morning        | 0     | Afternoon     | Mumbai           |   |
| 4 | Vistara  | Delhi       | Morning        | 0     | Morning       | Mumbai           |   |

|   | class   | duration | days_left | price |
|---|---------|----------|-----------|-------|
| 0 | Economy | 2.17     | 1         | 5953  |
| 1 | Economy | 2.33     | 1         | 5953  |
| 2 | Economy | 2.17     | 1         | 5956  |
| 3 | Economy | 2.25     | 1         | 5955  |
| 4 | Economy | 2.33     | 1         | 5955  |

```
# 파생변수 'route' = 'source_city' + 'destination_city'
```

```
df_cleaned['route']=df_cleaned['source_city']+'-'+df_cleaned['destination_city']
```

```
print("---<'route' 컬럼 생성 후>---")
```

```
print(df_cleaned.head())
```

```
---<'route' 컬럼 생성 후>---
```

|   | airline  | source_city | departure_time | stops | arrival_time  | destination_city | ₩ |
|---|----------|-------------|----------------|-------|---------------|------------------|---|
| 0 | SpiceJet | Delhi       | Evening        | 0     | Night         | Mumbai           |   |
| 1 | SpiceJet | Delhi       | Early_Morning  | 0     | Morning       | Mumbai           |   |
| 2 | AirAsia  | Delhi       | Early_Morning  | 0     | Early_Morning | Mumbai           |   |
| 3 | Vistara  | Delhi       | Morning        | 0     | Afternoon     | Mumbai           |   |
| 4 | Vistara  | Delhi       | Morning        | 0     | Morning       | Mumbai           |   |

|   | class   | duration | days_left | price | route        |
|---|---------|----------|-----------|-------|--------------|
| 0 | Economy | 2.17     | 1         | 5953  | Delhi-Mumbai |
| 1 | Economy | 2.33     | 1         | 5953  | Delhi-Mumbai |
| 2 | Economy | 2.17     | 1         | 5956  | Delhi-Mumbai |
| 3 | Economy | 2.25     | 1         | 5955  | Delhi-Mumbai |
| 4 | Economy | 2.33     | 1         | 5955  | Delhi-Mumbai |

## # 항공사별 네트워크 시각화

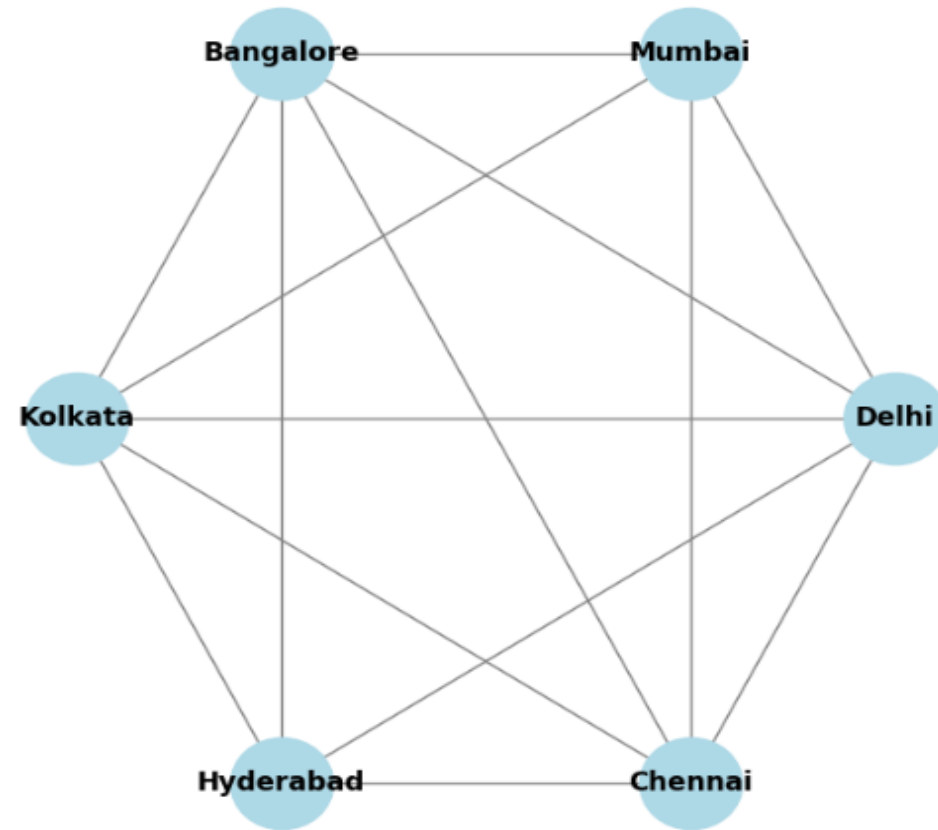
```
for airline in df['airline'].unique():
    subset = df[df["airline"] == airline]

    G = nx.Graph()

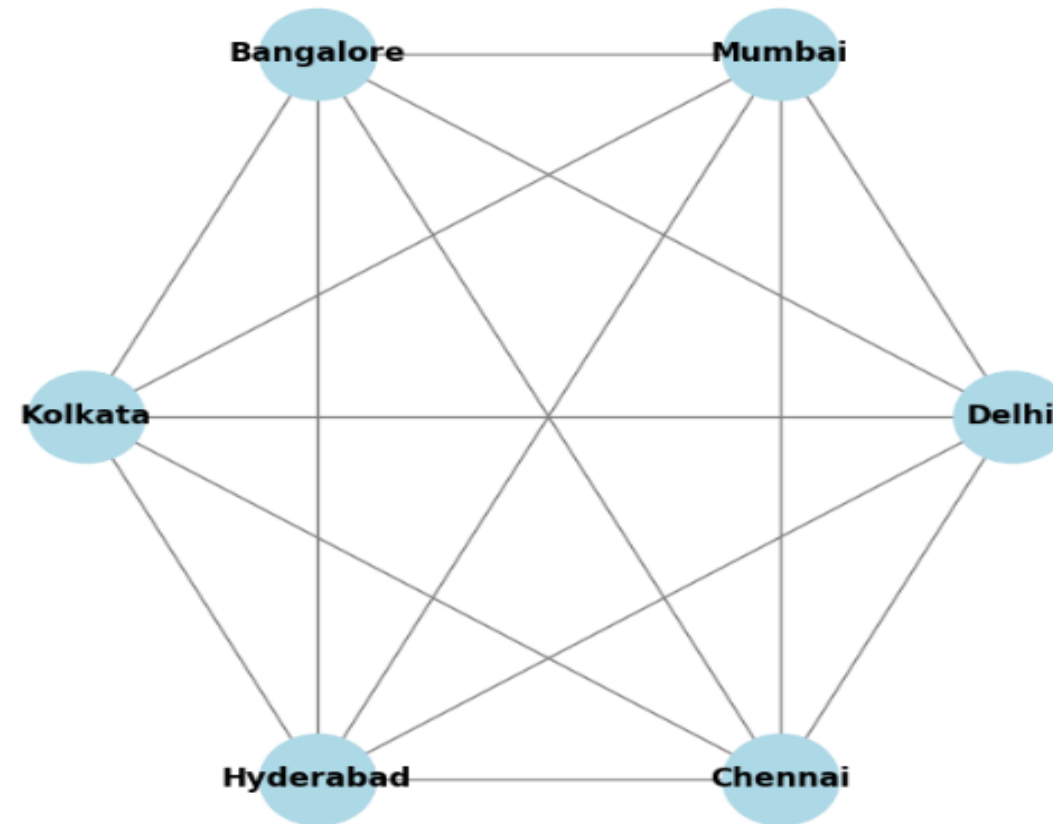
    # 엣지 추가 (출발도시 → 도착도시)
    for _, row in subset.iterrows():
        G.add_edge(row["source_city"], row["destination_city"])

    pos = nx.circular_layout(G)
    # 그래프 시각화
    plt.figure(figsize=(6,6))
    nx.draw(
        G, pos,
        with_labels=True,
        node_size=2000,
        node_color="lightblue",
        font_size=12,
        font_weight="bold",
        edge_color="gray"
    )
    plt.title(f"Airline Network: {airline}", fontsize=14)
    plt.show()
```

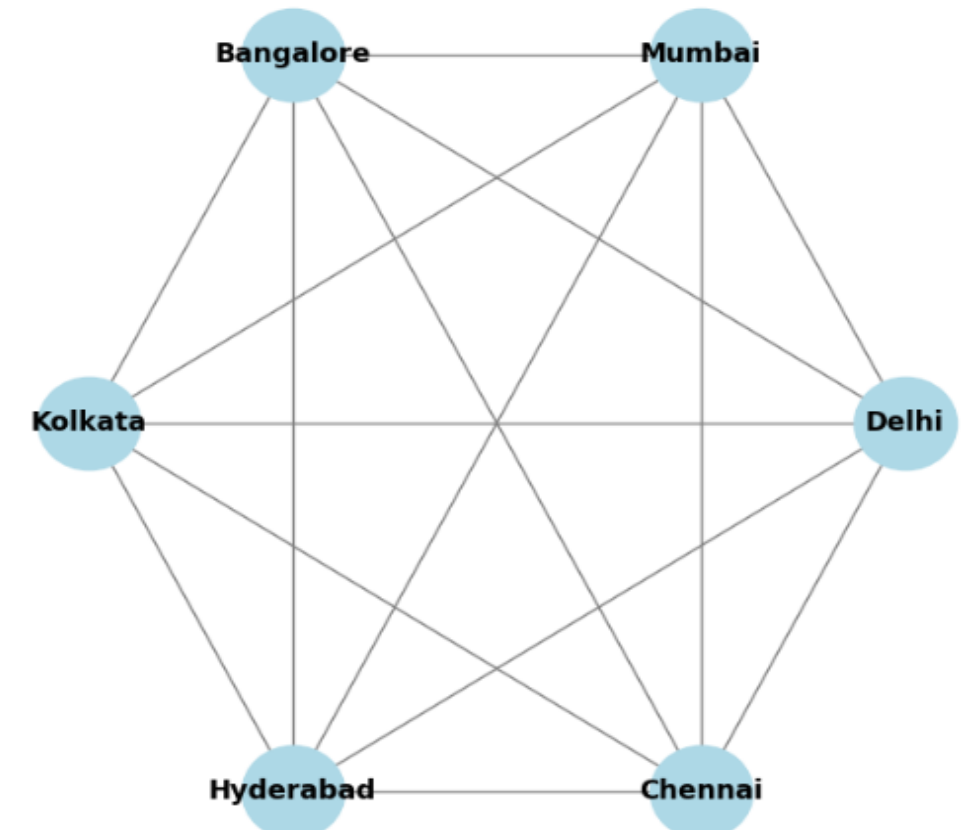
Airline Network: SpiceJet



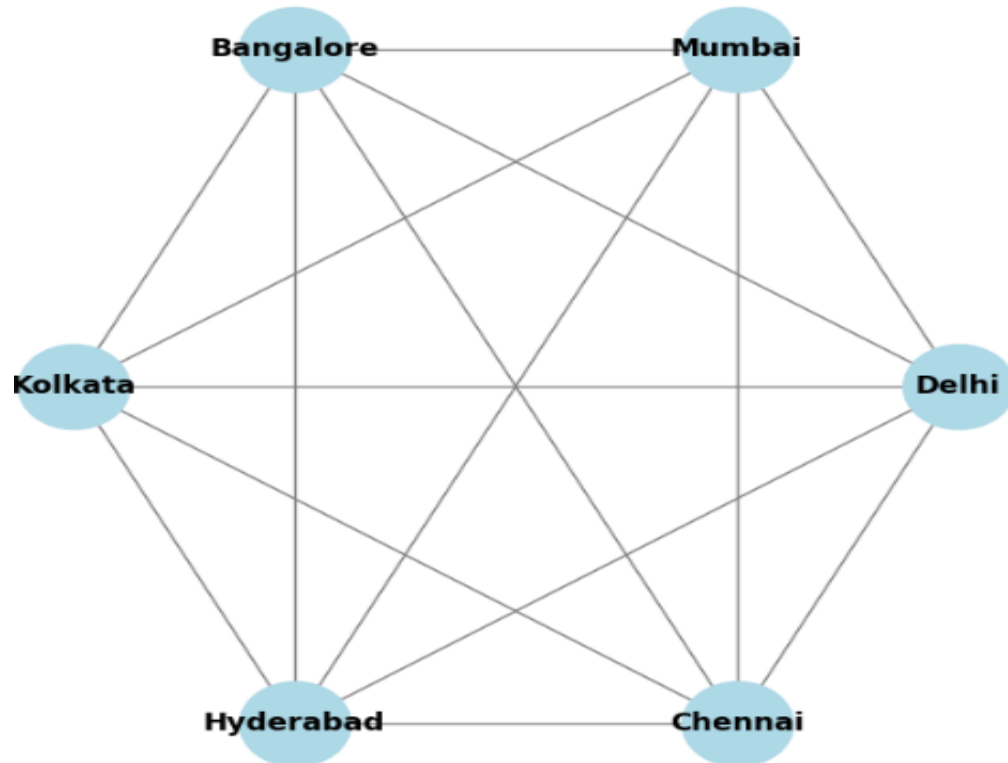
Airline Network: AirAsia



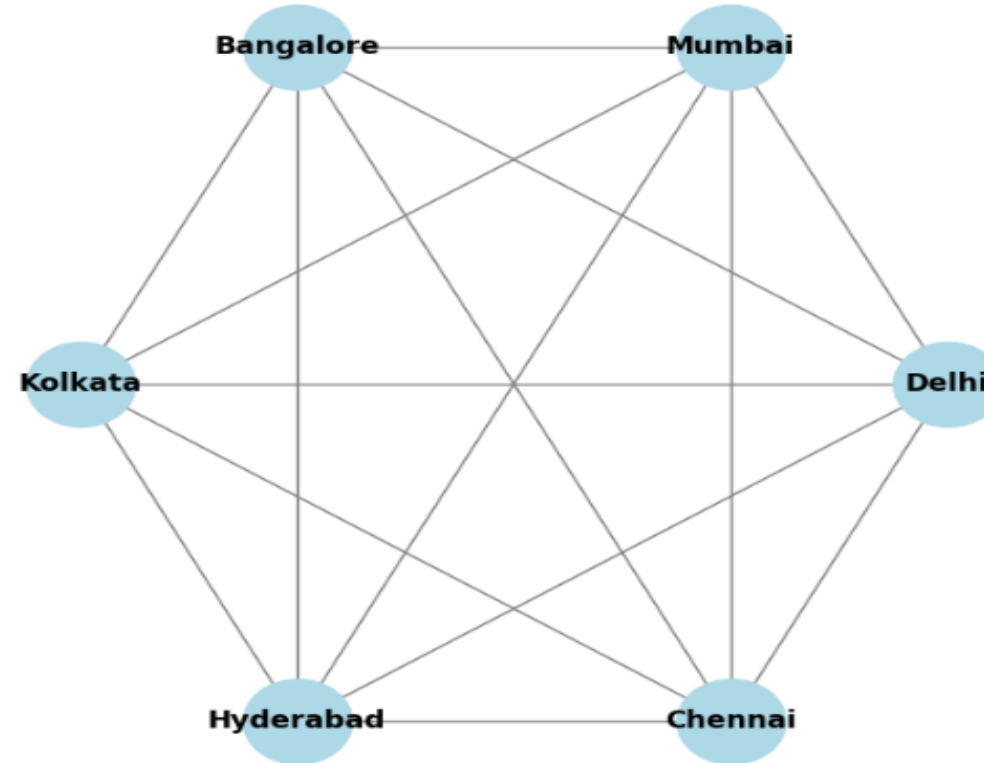
Airline Network: Vistara



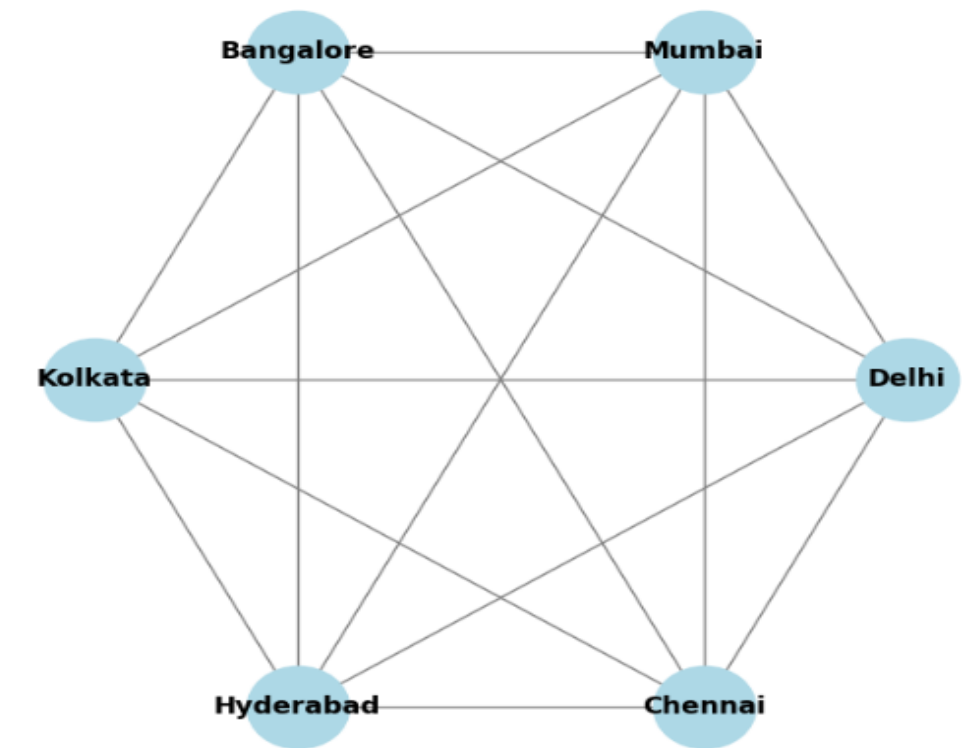
Airline Network: GO\_FIRST



Airline Network: Indigo

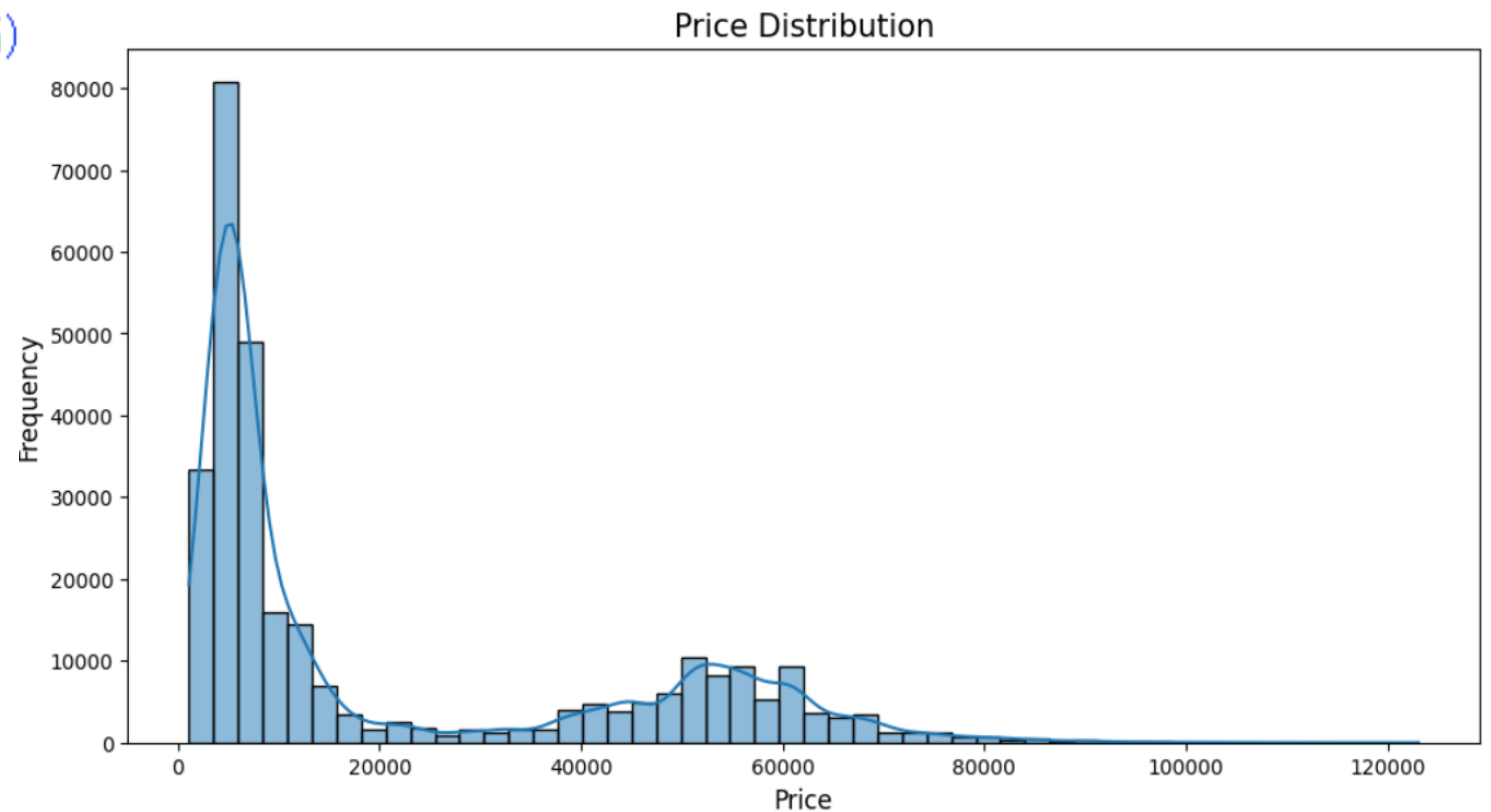


Airline Network: Air\_India



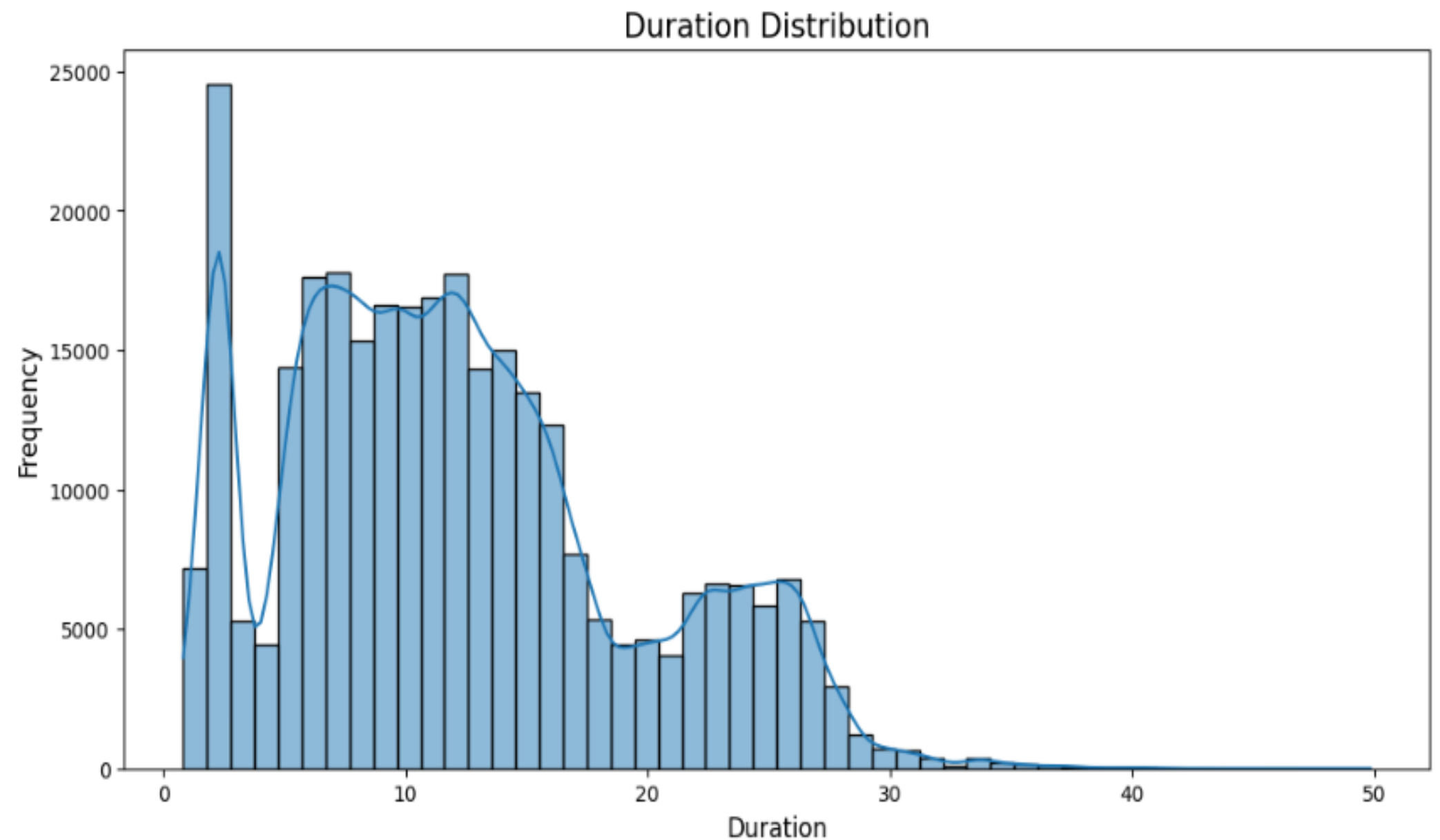
## # 가격('price') 분포 히스토그램으로 시각화

```
plt.figure(figsize=(12, 6))  
sns.histplot(data=df_cleaned, x='price', bins=50, kde=True)  
plt.title('Price Distribution', fontsize=15)  
plt.xlabel('Price', fontsize=12)  
plt.ylabel('Frequency', fontsize=12)  
plt.show()
```



# 시간('duration') 분포 히스토그램으로 시각화

```
plt.figure(figsize=(12, 6))  
sns.histplot(data=df_cleaned, x='duration', bins=50, kde=True)  
plt.title('Duration Distribution', fontsize=15)  
plt.xlabel('Duration', fontsize=12)  
plt.ylabel('Frequency', fontsize=12)  
plt.show()
```



# 'duration' 상위 1% 이상치 제거

```
threshold = df_cleaned["duration"].quantile(0.99)  
df_cleaned = df_cleaned[df_cleaned["duration"] <= threshold]
```

## # 로그변환해 파생변수 'price\_log', 'duration\_log' 컬럼 추가

```
df_cleaned['price_log']=np.log(df_cleaned['price'])
df_cleaned['duration_log']=np.log(df_cleaned['duration'])

print(df_cleaned.head())
```

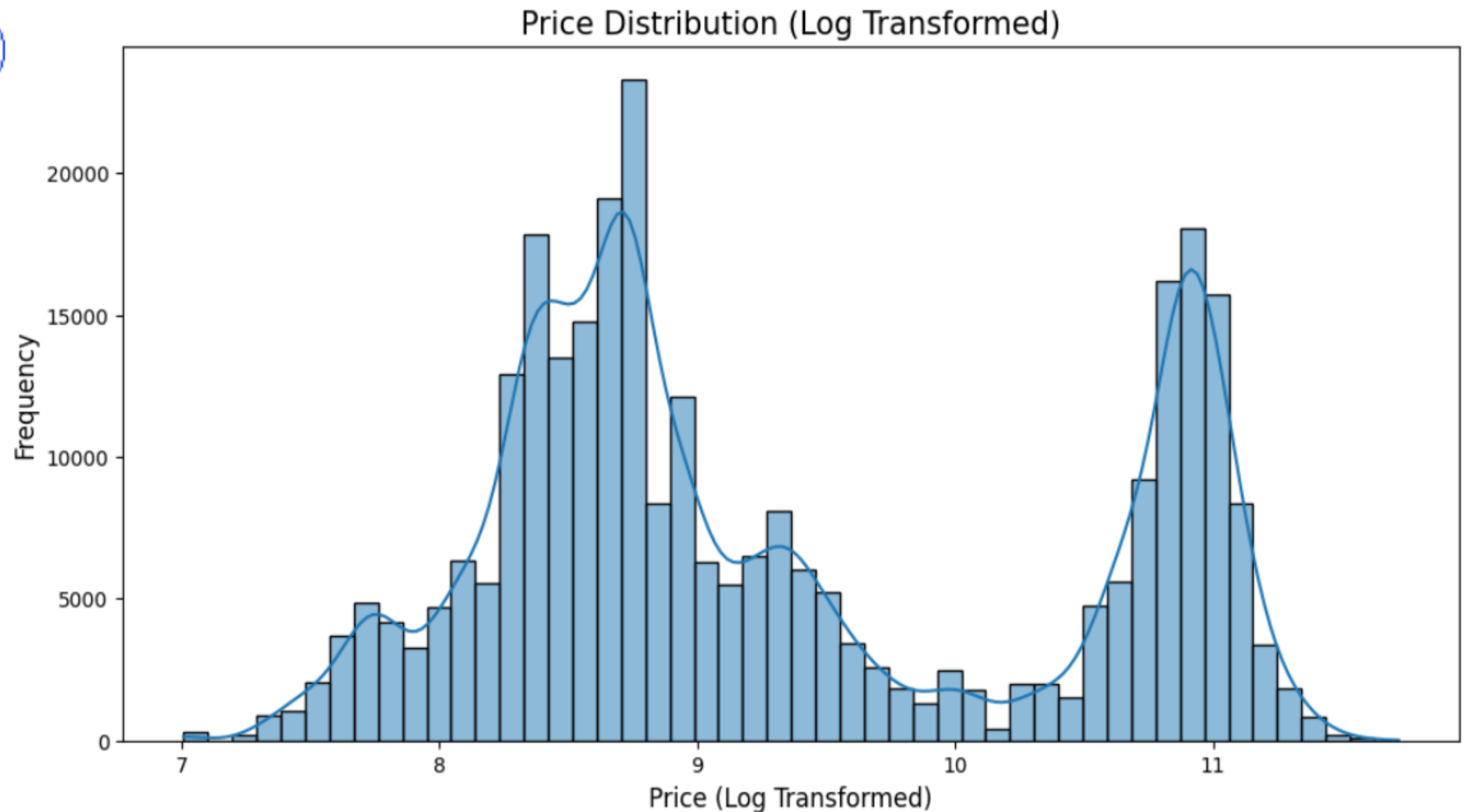
|   | airline  | source_city | departure_time | stops | arrival_time  | destination_city |
|---|----------|-------------|----------------|-------|---------------|------------------|
| 0 | SpiceJet | Delhi       | Evening        | 0     | Night         | Mumbai           |
| 1 | SpiceJet | Delhi       | Early_Morning  | 0     | Morning       | Mumbai           |
| 2 | AirAsia  | Delhi       | Early_Morning  | 0     | Early_Morning | Mumbai           |
| 3 | Vistara  | Delhi       | Morning        | 0     | Afternoon     | Mumbai           |
| 4 | Vistara  | Delhi       | Morning        | 0     | Morning       | Mumbai           |

|   | class   | duration | days_left | price | route        | price_log | duration_log |
|---|---------|----------|-----------|-------|--------------|-----------|--------------|
| 0 | Economy | 2.17     | 1         | 5953  | Delhi-Mumbai | 8.691651  | 0.774727     |
| 1 | Economy | 2.33     | 1         | 5953  | Delhi-Mumbai | 8.691651  | 0.845868     |
| 2 | Economy | 2.17     | 1         | 5956  | Delhi-Mumbai | 8.692154  | 0.774727     |
| 3 | Economy | 2.25     | 1         | 5955  | Delhi-Mumbai | 8.691986  | 0.810930     |
| 4 | Economy | 2.33     | 1         | 5955  | Delhi-Mumbai | 8.691986  | 0.845868     |

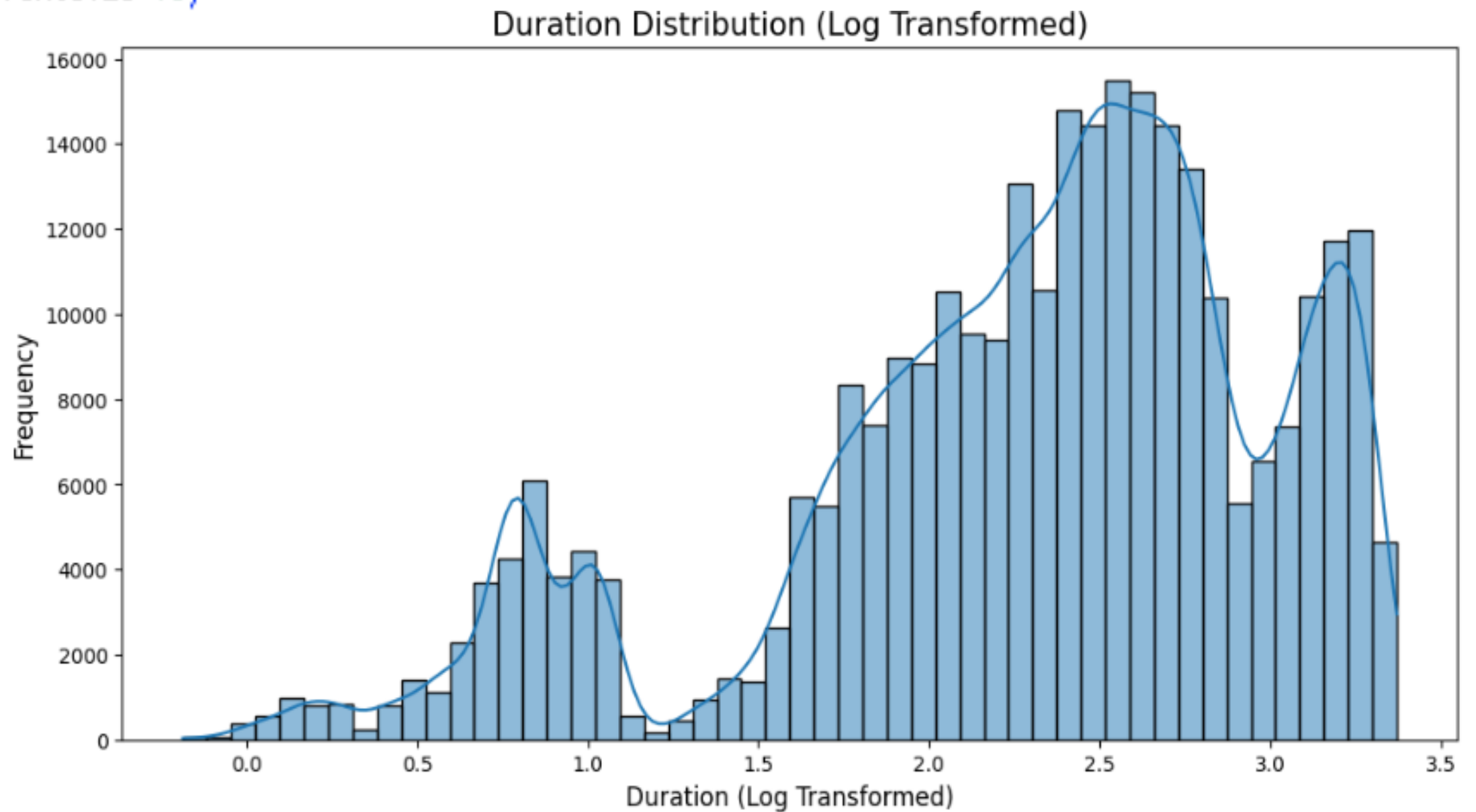
## # 로그변환하여 히스토그램으로 시각화 (가격)

```
plt.figure(figsize=(12, 6))  
sns.histplot(data=df_cleaned, x='price_log', bins=50, kde=True)  
plt.title('Price Distribution (Log Transformed)', fontsize=15)  
plt.xlabel('Price (Log Transformed)', fontsize=12)  
plt.ylabel('Frequency', fontsize=12)
```



## # 로그변환하여 히스토그램으로 시각화 (시간)

```
plt.figure(figsize=(12, 6))  
sns.histplot(data=df_cleaned, x='duration_log', bins=50, kde=True)  
plt.title('Duration Distribution (Log Transformed)', fontsize=15)  
plt.xlabel('Duration (Log Transformed)', fontsize=12)  
plt.ylabel('Frequency', fontsize=12)
```



## 원-핫 인코딩

```
# route 컬럼은 source_city와 destination_city의 정보가 합쳐져 있으므로 제외하고 각각 인코딩  
categorical_features = ['airline', 'source_city', 'destination_city', 'departure_time', 'arrival_time', 'class']
```

# 선형/비선형 모델용으로 나눠 각각 인코딩 (drop\_first 옵션 차이)

```
df_encoded_linear = pd.get_dummies(df_cleaned, columns=categorical_features, drop_first=True) # 선형  
df_encoded = pd.get_dummies(df_cleaned, columns=categorical_features, drop_first=False) # 비선형
```

# 인코딩 후 데이터 샘플 및 정보 확인

```
print(df_encoded.head())  
df_encoded.info()
```

# 원-핫 인코딩

## # 인코딩 후 데이터 샘플 및 정보 확인

```
print("\n--- [인코딩 후 데이터 정보 확인] ---")
df_encoded_linear.info()
df_encoded.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 297177 entries, 0 to 300152
Data columns (total 39 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   stops                                297177 non-null  int64
1   duration                            297177 non-null  float64
2   days_left                           297177 non-null  int64
3   price                               297177 non-null  int64
4   route                               297177 non-null  object
5   price_log                           297177 non-null  float64
6   duration_log                        297177 non-null  float64
7   airline_AirAsia                     297177 non-null  bool
8   airline_AirIndia                    297177 non-null  bool
9   airline_GO_FIRST                    297177 non-null  bool
10  airline_Indigo                       297177 non-null  bool
11  airline_SpiceJet                     297177 non-null  bool
12  airline_Vistara                      297177 non-null  bool
13  source_city_Bangalore                297177 non-null  bool
14  source_city_Chennai                 297177 non-null  bool
15  source_city_Delhi                    297177 non-null  bool
16  source_city_Hyderabad                297177 non-null  bool
17  source_city_Kolkata                  297177 non-null  bool
18  source_city_Mumbai                   297177 non-null  bool
19  destination_city_Bangalore            297177 non-null  bool
20  destination_city_Chennai              297177 non-null  bool
21  destination_city_Delhi                297177 non-null  bool
22  destination_city_Hyderabad            297177 non-null  bool
23  destination_city_Kolkata              297177 non-null  bool
24  destination_city_Mumbai               297177 non-null  bool
25  departure_time_Afternoon              297177 non-null  bool
26  departure_time_Early_Morning          297177 non-null  bool
27  departure_time_Evening                297177 non-null  bool
28  departure_time_Late_Night              297177 non-null  bool
29  departure_time_Morning                297177 non-null  bool
30  departure_time_Night                  297177 non-null  bool
31  arrival_time_Afternoon                297177 non-null  bool
32  arrival_time_Early_Morning            297177 non-null  bool
33  arrival_time_Evening                  297177 non-null  bool
34  arrival_time_Late_Night               297177 non-null  bool
35  arrival_time_Morning                  297177 non-null  bool
36  arrival_time_Night                    297177 non-null  bool
37  class_Business                       297177 non-null  bool
38  class_Economy                        297177 non-null  bool
dtypes: bool(32), float64(3), int64(3), object(1)
memory usage: 27.2+ MB
```

--- [인코딩 후 데이터 정보 확인] ---

```
<class 'pandas.core.frame.DataFrame'>
```

Index: 297177 entries, 0 to 300152

Data columns (total 33 columns):

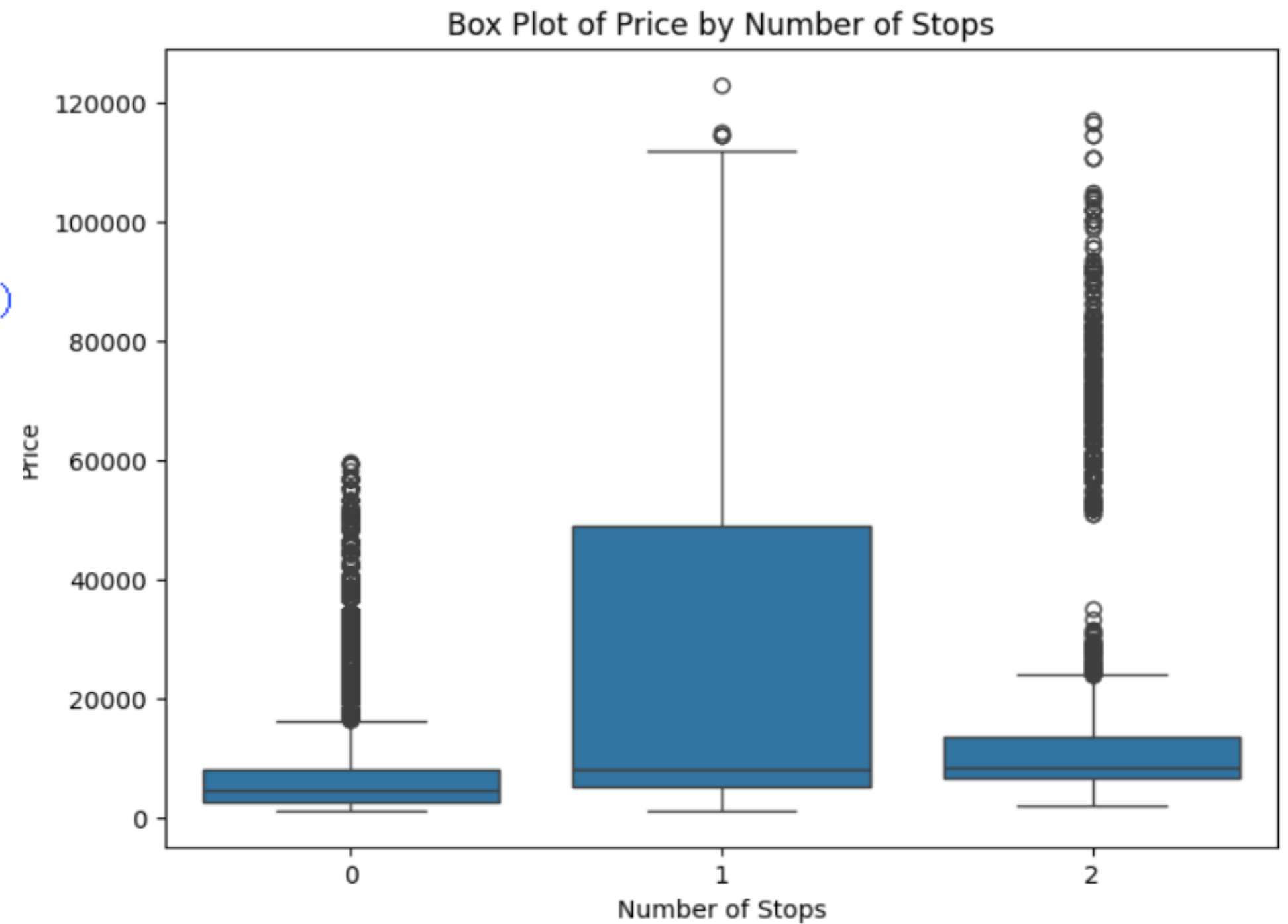
| #  | Column                       | Non-Null Count  | Dtype   |
|----|------------------------------|-----------------|---------|
| 0  | stops                        | 297177 non-null | int64   |
| 1  | duration                     | 297177 non-null | float64 |
| 2  | days_left                    | 297177 non-null | int64   |
| 3  | price                        | 297177 non-null | int64   |
| 4  | route                        | 297177 non-null | object  |
| 5  | price_log                    | 297177 non-null | float64 |
| 6  | duration_log                 | 297177 non-null | float64 |
| 7  | airline_AirIndia             | 297177 non-null | bool    |
| 8  | airline_GO_FIRST             | 297177 non-null | bool    |
| 9  | airline_Indigo               | 297177 non-null | bool    |
| 10 | airline_SpiceJet             | 297177 non-null | bool    |
| 11 | airline_Vistara              | 297177 non-null | bool    |
| 12 | source_city_Chennai          | 297177 non-null | bool    |
| 13 | source_city_Delhi            | 297177 non-null | bool    |
| 14 | source_city_Hyderabad        | 297177 non-null | bool    |
| 15 | source_city_Kolkata          | 297177 non-null | bool    |
| 16 | source_city_Mumbai           | 297177 non-null | bool    |
| 17 | destination_city_Chennai     | 297177 non-null | bool    |
| 18 | destination_city_Delhi       | 297177 non-null | bool    |
| 19 | destination_city_Hyderabad   | 297177 non-null | bool    |
| 20 | destination_city_Kolkata     | 297177 non-null | bool    |
| 21 | destination_city_Mumbai      | 297177 non-null | bool    |
| 22 | departure_time_Early_Morning | 297177 non-null | bool    |
| 23 | departure_time_Evening       | 297177 non-null | bool    |
| 24 | departure_time_Late_Night    | 297177 non-null | bool    |
| 25 | departure_time_Morning       | 297177 non-null | bool    |
| 26 | departure_time_Night         | 297177 non-null | bool    |
| 27 | arrival_time_Early_Morning   | 297177 non-null | bool    |
| 28 | arrival_time_Evening         | 297177 non-null | bool    |
| 29 | arrival_time_Late_Night      | 297177 non-null | bool    |
| 30 | arrival_time_Morning         | 297177 non-null | bool    |
| 31 | arrival_time_Night           | 297177 non-null | bool    |
| 32 | class_Economy                | 297177 non-null | bool    |

dtypes: bool(26), float64(3), int64(3), object(1)

memory usage: 25.5+ MB

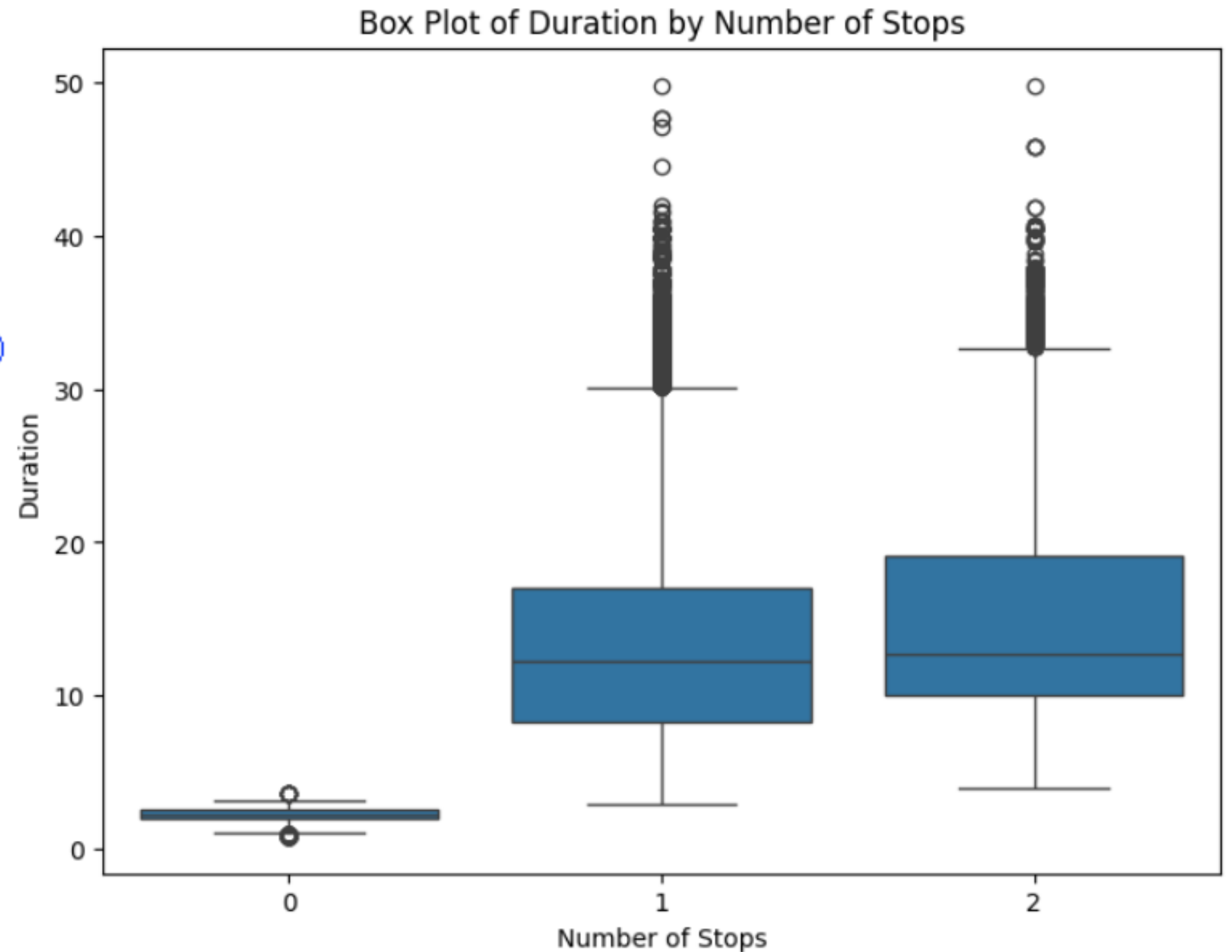
# 경유 횟수('stops')에 따른 가격('price')의 분포를 박스 플롯으로 시각화

```
plt.figure(figsize=(8, 6))
sns.boxplot(data=df_cleaned, x='stops', y='price')
plt.title('Box Plot of Price by Number of Stops')
plt.xlabel('Number of Stops')
plt.ylabel('Price')
plt.show()
```



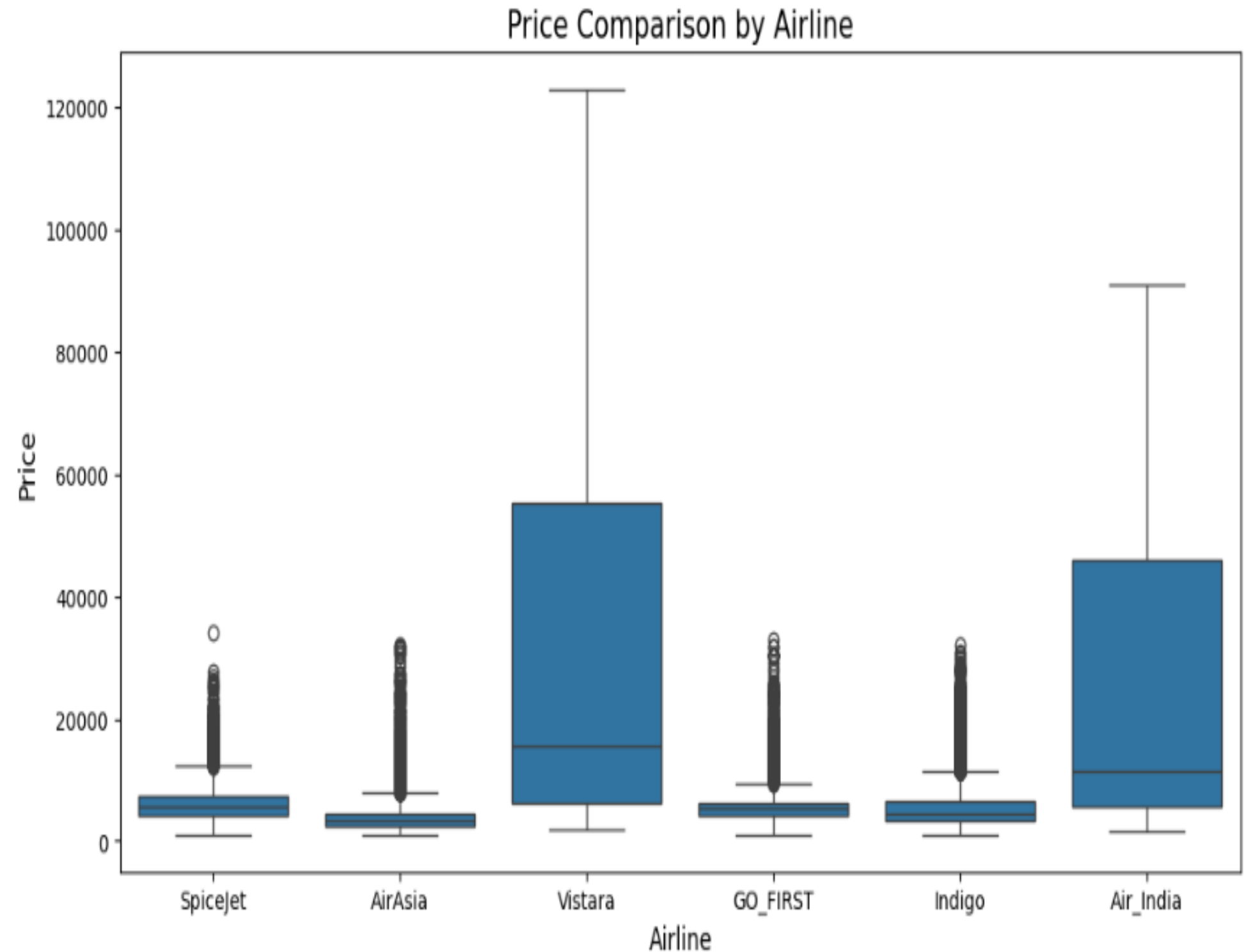
# 경유 횟수('stops')에 따른 시간('duration')의 분포를 박스 플롯으로 시각화

```
plt.figure(figsize=(8, 6))  
sns.boxplot(data=df_cleaned, x='stops', y='duration')  
plt.title('Box Plot of Duration by Number of Stops')  
plt.xlabel('Number of Stops')  
plt.ylabel('Duration')  
plt.show()
```



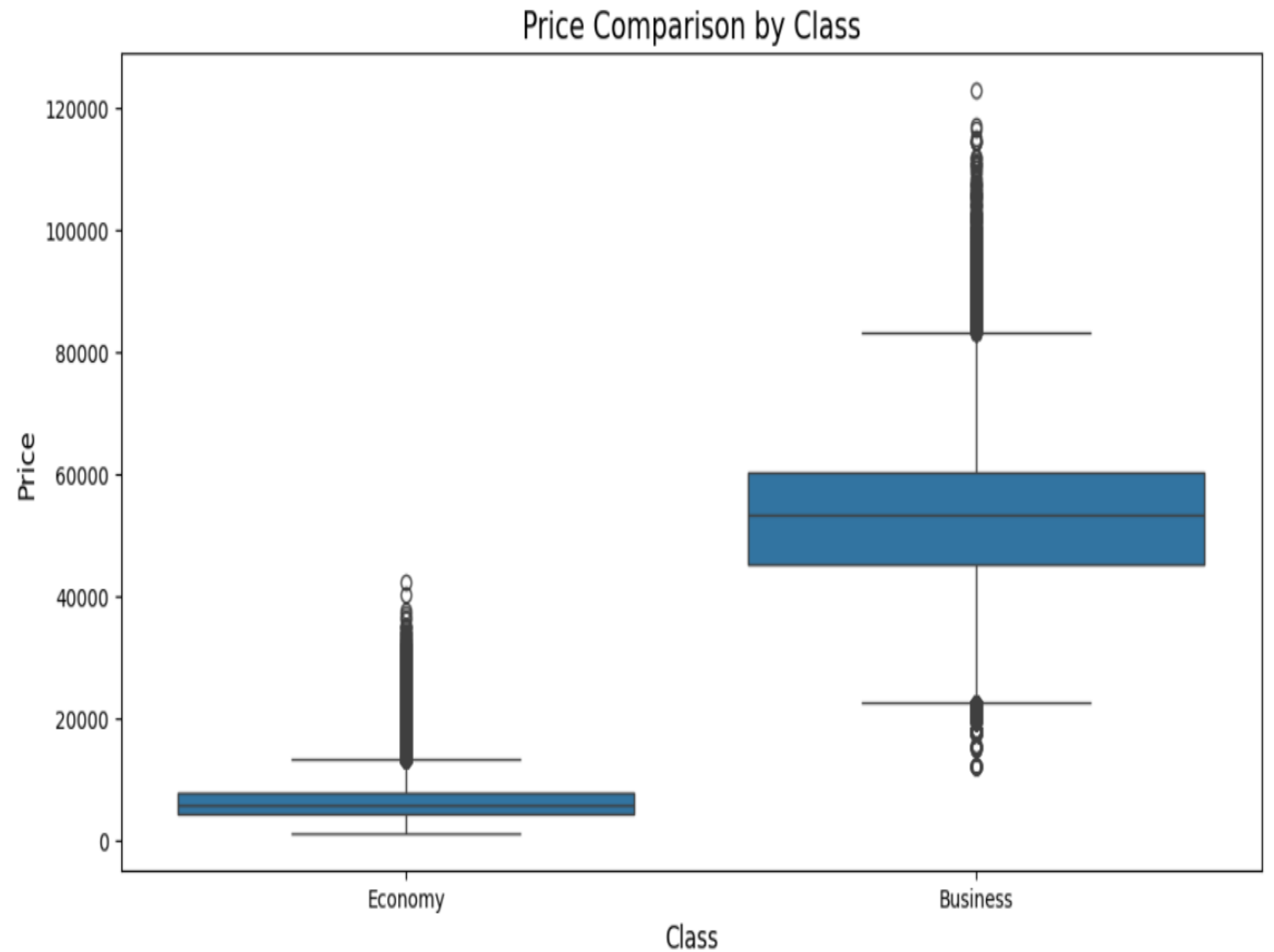
## # 항공사별 가격 비교 ('airline')

```
plt.figure(figsize=(12, 6))
sns.boxplot(data=df_cleaned, x='airline', y='price')
plt.title('Price Comparison by Airline', fontsize=15)
plt.xlabel('Airline', fontsize=12)
plt.ylabel('Price', fontsize=12)
plt.show()
```



## # 좌석 등급별 가격 비교 ('class')

```
plt.figure(figsize=(12, 6))  
sns.boxplot(data=df_cleaned, x='class', y='price')  
plt.title('Price Comparison by Class', fontsize=15)  
plt.xlabel('Class', fontsize=12)  
plt.ylabel('Price', fontsize=12)  
plt.show()
```

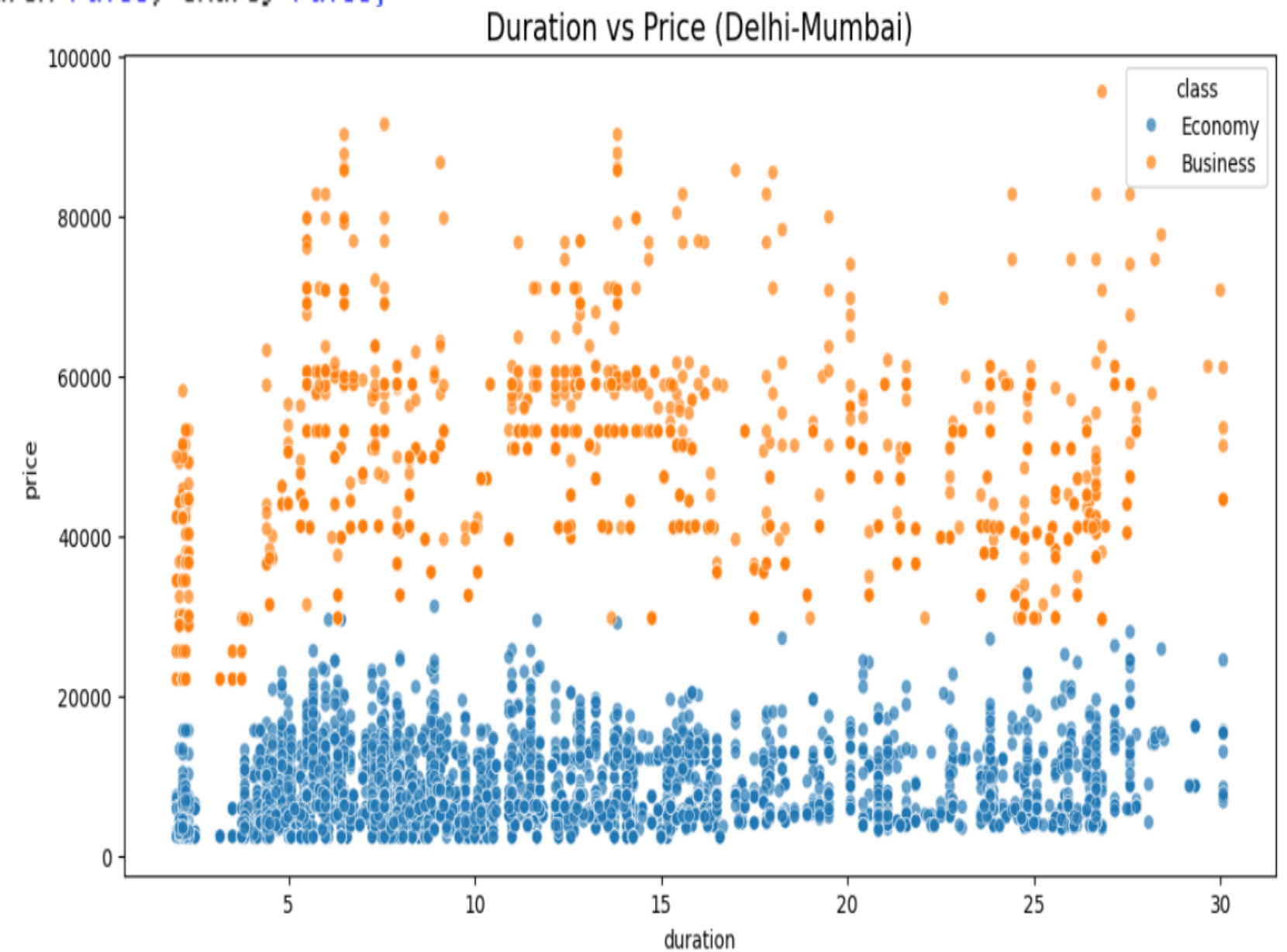


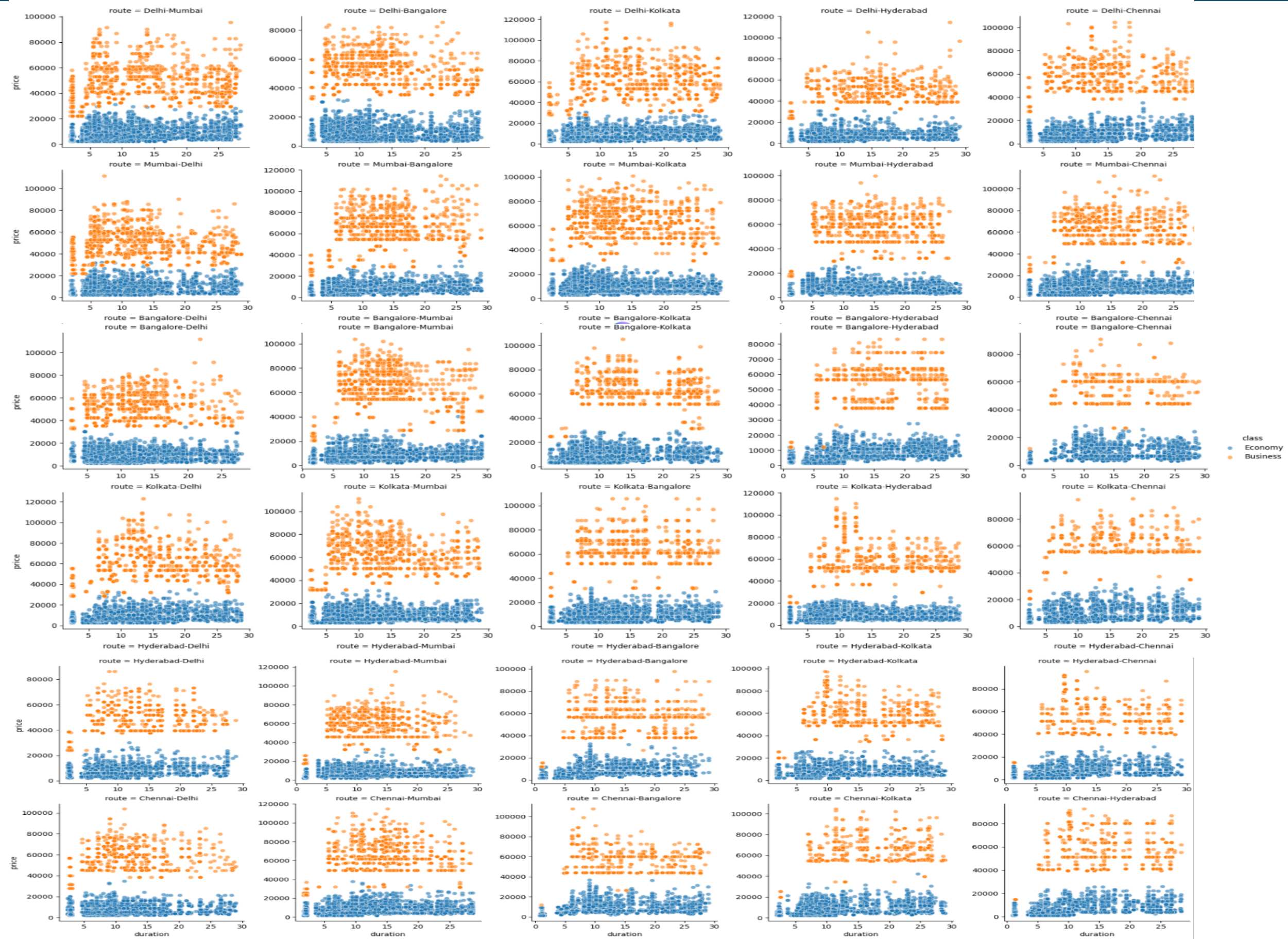
## # 비행시간('duration')과 가격('price')의 관계

좌석 등급(class)에 따라 점의 색상 구분 (hue='class')

```
# FacetGrid로 route 별로 subplot 생성
g = sns.FacetGrid(df_cleaned, col="route", hue="class", col_wrap=5, height=4, sharex=False, sharey=False)
g.map(sns.scatterplot, "duration", "price", alpha=0.6)
g.add_legend()
g.fig.subplots_adjust(top=0.9)
g.fig.suptitle("Duration vs Price by Route and Class", fontsize=16)

plt.show()
```





## # 상관 / 연관 분석

수치 ↔ 수치 상관 (피어슨/스피어만)

```
num_cols = ["price", "price_log", "duration", "duration_log", "days_left", "stops"]

pearson_corr = df_cleaned[num_cols].corr(method="pearson")
spearman_corr = df_cleaned[num_cols].corr(method="spearman")

print("=== Pearson 상관 ==="); display(pearson_corr.round(3))
print("\n")
print("=== Spearman 상관 ==="); display(spearman_corr.round(3))
print("\n")

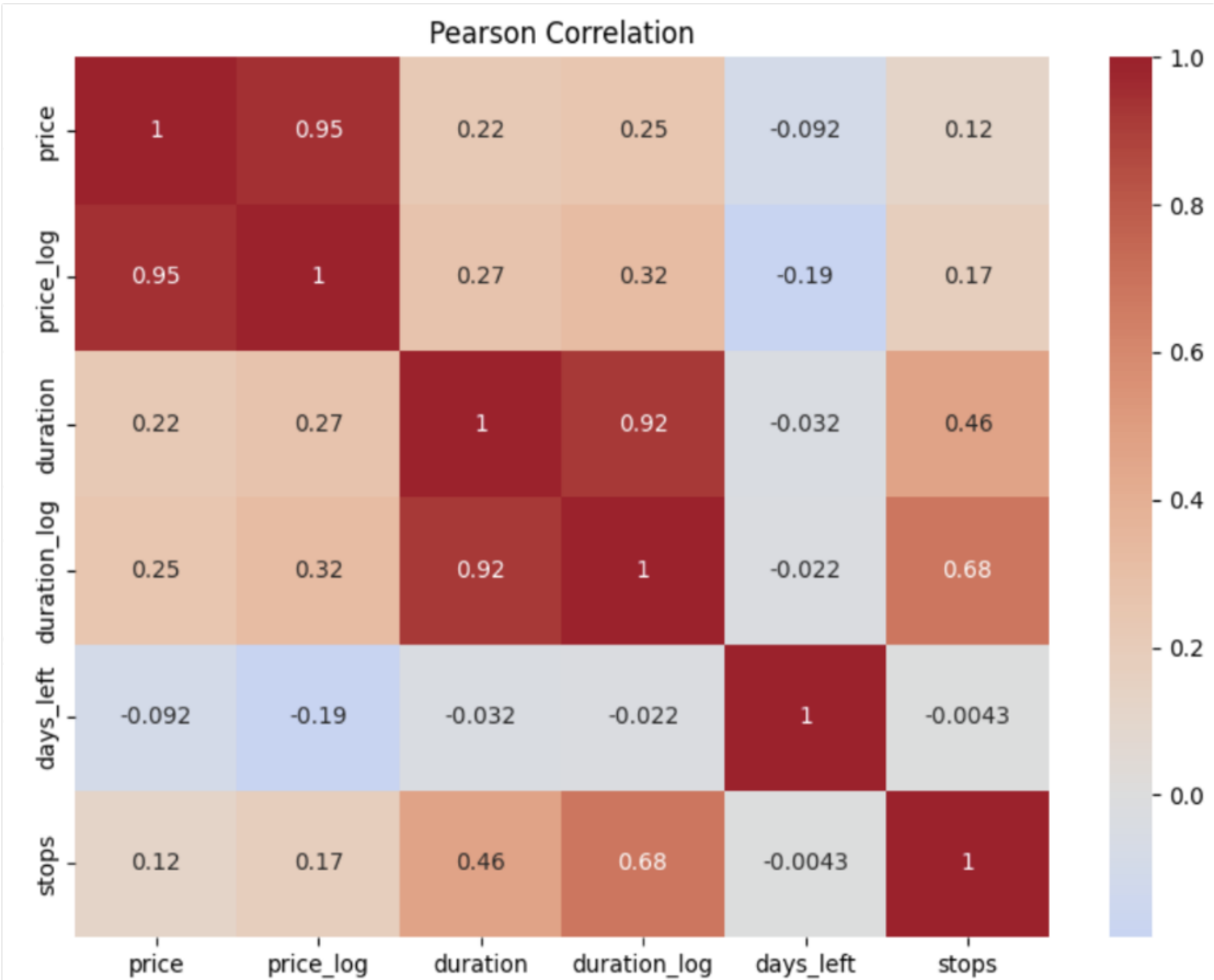
plt.figure(figsize=(8,6))
sns.heatmap(pearson_corr, annot=True, cmap="coolwarm", center=0)
plt.title("Pearson Correlation")
plt.tight_layout()
plt.show()
```

=== Pearson 상관 ===

|              | price  | price_log | duration | duration_log | days_left | stops  |
|--------------|--------|-----------|----------|--------------|-----------|--------|
| price        | 1.000  | 0.947     | 0.217    | 0.254        | -0.092    | 0.124  |
| price_log    | 0.947  | 1.000     | 0.272    | 0.316        | -0.193    | 0.172  |
| duration     | 0.217  | 0.272     | 1.000    | 0.923        | -0.032    | 0.464  |
| duration_log | 0.254  | 0.316     | 0.923    | 1.000        | -0.022    | 0.679  |
| days_left    | -0.092 | -0.193    | -0.032   | -0.022       | 1.000     | -0.004 |
| stops        | 0.124  | 0.172     | 0.464    | 0.679        | -0.004    | 1.000  |

=== Spearman 상관 ===

|              | price  | price_log | duration | duration_log | days_left | stops  |
|--------------|--------|-----------|----------|--------------|-----------|--------|
| price        | 1.000  | 1.000     | 0.319    | 0.319        | -0.266    | 0.224  |
| price_log    | 1.000  | 1.000     | 0.319    | 0.319        | -0.266    | 0.224  |
| duration     | 0.319  | 0.319     | 1.000    | 1.000        | -0.029    | 0.510  |
| duration_log | 0.319  | 0.319     | 1.000    | 1.000        | -0.029    | 0.510  |
| days_left    | -0.266 | -0.266    | -0.029   | -0.029       | 1.000     | -0.004 |
| stops        | 0.224  | 0.224     | 0.510    | 0.510        | -0.004    | 1.000  |



# route 디메안 상관 (노선별 평균으로 노선 효과 제거 후 price↔duration)

```
p_dm = df_cleaned["price"] - df_cleaned.groupby("route")["price"].transform("mean")
t_dm = df_cleaned["duration"] - df_cleaned.groupby("route")["duration"].transform("mean")
r_dm = np.corrcoef(p_dm, t_dm)[0,1]
print(f"노선 효과 제거 후 price↔duration 상관(피어슨): {r_dm:.3f}")
```

노선 효과 제거 후 price↔duration 상관(피어슨): 0.211

## # 범주↔수치 (ANOVA + Kruskal-Wallis)

#anova

```
def anova_effect_size(df, num_col, cat_col):  
    groups = [g[num_col].dropna().values for _, g in df.groupby(cat_col) if len(g)]  
    if len(groups) < 2: return np.nan, np.nan, np.nan  
  
    f, p = stats.f_oneway(*groups) # 일원배치분산분석  
    grand = df[num_col].mean() # 전체 평균  
    ssb = sum(len(g)*(np.mean(g)-grand)**2 for g in groups) # 그룹별 제곱합  
    sst = sum(((g - grand)**2).sum() for g in groups) # 전체 제곱합  
    eta2 = ssb/sst if sst>0 else np.nan # 설명력  
    return f, p, eta2
```

#kruskal

```
def kruskal_test(df, num_col, cat_col):  
    groups = [g[num_col].dropna().values for _, g in df.groupby(cat_col) if len(g)]  
    if len(groups) < 2: return np.nan, np.nan  
  
    h, p = stats.kruskal(*groups)  
    return h, p
```

```
for cat_col in ["airline", "class", "departure_time", "arrival_time"]:  
    for num_col in ["price", "duration"]:  
        f, p, eta2 = anova_effect_size(df_cleaned, num_col, cat_col)  
        print(f"[ANOVA] {num_col} ~ {cat_col}: F={f:.2f}, p={p:.3e}, eta={eta2:.3f}")  
  
        h, p_kw = kruskal_test(df_cleaned, num_col, cat_col)  
        print(f"[Kruskal-Wallis] H={h:.2f}, p={p_kw:.3e}")
```

```
[ANOVA] price ~ airline: F=17233.26, p=0.000e+00, eta=0.225  
[Kruskal-Wallis] H=93762.44, p=0.000e+00  
[ANOVA] duration ~ airline: F=15622.33, p=0.000e+00, eta=0.208  
[Kruskal-Wallis] H=69065.64, p=0.000e+00  
[ANOVA] price ~ class: F=2175189.39, p=0.000e+00, eta=0.880  
[Kruskal-Wallis] H=191483.38, p=0.000e+00  
[ANOVA] duration ~ class: F=7362.14, p=0.000e+00, eta=0.024  
[Kruskal-Wallis] H=8864.85, p=0.000e+00  
[ANOVA] price ~ departure_time: F=326.88, p=0.000e+00, eta=0.005  
[Kruskal-Wallis] H=1938.02, p=0.000e+00  
[ANOVA] duration ~ departure_time: F=2766.17, p=0.000e+00, eta=0.044  
[Kruskal-Wallis] H=14876.98, p=0.000e+00  
[ANOVA] price ~ arrival_time: F=1021.47, p=0.000e+00, eta=0.017  
[Kruskal-Wallis] H=8905.46, p=0.000e+00  
[ANOVA] duration ~ arrival_time: F=1977.89, p=0.000e+00, eta=0.032  
[Kruskal-Wallis] H=11586.51, p=0.000e+00
```

## # 범주↔범주 (Cramér's V)

```
def cramers_v(ct):
    chi2 = stats.chi2_contingency(ct)[0]
    n = ct.values.sum()
    phi2 = chi2/n
    r,k = ct.shape
    # bias-corrected Cramér's V
    phi2corr = max(0, phi2 - (k-1)*(r-1)/(n-1))
    rcorr = r - (r-1)**2/(n-1)
    kcorr = k - (k-1)**2/(n-1)
    denom = min((kcorr-1),(rcorr-1))
    return np.sqrt(phi2corr/denom) if denom>0 else np.nan

pairs = [
    ("airline", "class"),
    ("airline", "departure_time"),
    ("airline", "stops")
]

for c1, c2 in pairs:
    ct = pd.crosstab(df_cleaned[c1], df_cleaned[c2])
    v = cramers_v(ct)
    print(f"Cramér's V ({c1} vs {c2}) = {v:.3f}")
    display(ct)
    print("\n")
```

Cramér's V (airline vs class) = 0.449

| class     | Business | Economy |
|-----------|----------|---------|
| airline   |          |         |
| AirAsia   | 0        | 16098   |
| Air_India | 32898    | 47994   |
| GO_FIRST  | 0        | 23173   |
| Indigo    | 0        | 43120   |
| SpiceJet  | 0        | 9011    |
| Vistara   | 60589    | 67270   |

Cramér's V (airline vs stops) = 0.174

| stops     | 0     | 1      | 2    |
|-----------|-------|--------|------|
| airline   |       |        |      |
| AirAsia   | 2434  | 11418  | 2246 |
| Air_India | 6409  | 71004  | 3479 |
| GO_FIRST  | 3223  | 19545  | 405  |
| Indigo    | 11216 | 31166  | 738  |
| SpiceJet  | 2462  | 6549   | 0    |
| Vistara   | 10260 | 111181 | 6418 |

Cramér's V (airline vs departure\_time) = 0.081

| departure_time | Afternoon | Early_Morning | Evening | Late_Night | Morning | Night |
|----------------|-----------|---------------|---------|------------|---------|-------|
| airline        |           |               |         |            |         |       |
| AirAsia        | 3078      | 3692          | 3657    | 143        | 2348    | 3180  |
| Air_India      | 10876     | 19867         | 18626   | 291        | 17768   | 13464 |
| GO_FIRST       | 4942      | 6103          | 4904    | 146        | 4116    | 2962  |
| Indigo         | 10155     | 8184          | 8460    | 726        | 11491   | 4104  |
| SpiceJet       | 1193      | 2728          | 2031    | 0          | 1519    | 1540  |
| Vistara        | 17550     | 26216         | 27424   | 0          | 33904   | 22765 |

## # Business 클래스를 운영하는 항공사 목록 확인

```
business_airlines = df_cleaned.loc[df_cleaned["class"] == "Business", "airline"].unique()

print("Business 클래스가 있는 항공사 목록:")
print(business_airlines)
```

---

```
Business 클래스가 있는 항공사 목록:
['Air_India' 'Vistara']
```

## # 추가 피처엔지니어링

```
# ===== 추가 피처엔지니어링 =====
```

```
# (1) 레드아이/저녁 도착 플래그
```

```
df_cleaned["is_redeye_from_dep"] = df_cleaned["departure_time"].isin(["Night", "Early_Morning"]).astype(int)
```

```
df_cleaned["is_evening_arrival"] = df_cleaned["arrival_time"].isin(["Evening", "Night"]).astype(int)
```

```
# (2) 노선별 항공사 경쟁강도(HHI)
```

```
_cnt = (df_cleaned.groupby(["route", "airline"]).size().rename("n").reset_index()) # 운항수
```

```
_cnt["share"] = _cnt["n"] / _cnt.groupby("route")["n"].transform("sum") # 노선별 점유율
```

```
route_hhi = _cnt.groupby("route")["share"].apply(lambda s: (s**2).sum()).rename("route_hhi").reset_index()
```

```
df_cleaned = df_cleaned.merge(route_hhi, on="route", how="left")
```

## # df\_encoded에 파생 수치 피처 붙이기

```
df_encoded = df_encoded.reset_index(drop=True)
df_encoded_linear = df_encoded_linear.reset_index(drop=True)
df_cleaned = df_cleaned.reset_index(drop=True)

to_add = ["is_redeye_from_dep", "is_evening_arrival", "route_hhi"]
df_encoded_linear[to_add] = df_cleaned[to_add]
df_encoded[to_add] = df_cleaned[to_add]

df_encoded.info()
print("\n")
df_encoded_linear.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 297177 entries, 0 to 297176
Data columns (total 42 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   stops                                297177 non-null  int64
1   duration                             297177 non-null  float64
2   days_left                            297177 non-null  int64
3   price                                297177 non-null  int64
4   route                                297177 non-null  object
5   price_log                            297177 non-null  float64
6   duration_log                         297177 non-null  float64
7   airline_AirAsia                      297177 non-null  bool
8   airline_Air_India                   297177 non-null  bool
9   airline_GO_FIRST                    297177 non-null  bool
10  airline_Indigo                      297177 non-null  bool
11  airline_SpiceJet                    297177 non-null  bool
12  airline_Vistara                     297177 non-null  bool
13  source_city_Bangalore               297177 non-null  bool
14  source_city_Chennai                297177 non-null  bool
15  source_city_Delhi                   297177 non-null  bool
16  source_city_Hyderabad               297177 non-null  bool
17  source_city_Kolkata                 297177 non-null  bool
18  source_city_Mumbai                  297177 non-null  bool
19  destination_city_Bangalore          297177 non-null  bool
20  destination_city_Chennai            297177 non-null  bool
21  destination_city_Delhi              297177 non-null  bool
22  destination_city_Hyderabad          297177 non-null  bool
23  destination_city_Kolkata            297177 non-null  bool
24  destination_city_Mumbai             297177 non-null  bool
25  departure_time_Afternoon            297177 non-null  bool
26  departure_time_Early_Morning        297177 non-null  bool
27  departure_time_Evening               297177 non-null  bool
28  departure_time_Late_Night           297177 non-null  bool
29  departure_time_Morning              297177 non-null  bool
30  departure_time_Night                297177 non-null  bool
31  arrival_time_Afternoon              297177 non-null  bool
32  arrival_time_Early_Morning          297177 non-null  bool
33  arrival_time_Evening                297177 non-null  bool
34  arrival_time_Late_Night             297177 non-null  bool
35  arrival_time_Morning                297177 non-null  bool
36  arrival_time_Night                  297177 non-null  bool
37  class_Business                      297177 non-null  bool
38  class_Economy                      297177 non-null  bool
39  is_redeye_from_dep                  297177 non-null  int64
40  is_evening_arrival                  297177 non-null  int64
41  route_hhi                           297177 non-null  float64
```

## # 모델 설계 및 데이터셋 예측

```
df_encoded.drop(columns=["price", "duration", "route"], inplace=True)
df_encoded_linear.drop(columns=["price", "duration", "route"], inplace=True)

# ===== X / y 구성 =====
targets = ["price_log", "duration_log"]
X = df_encoded.drop(columns=targets, errors="ignore").copy()
X_linear = df_encoded_linear.drop(columns=targets, errors="ignore").copy()
y = df_encoded[targets].copy()

print(" Feature design (with one-hot ready) complete")
print(" - X shape:", X.shape)
print(" - X_linear shape:", X_linear.shape)
print(" - y shape:", y.shape)
print(" - targets:", targets)
```

## # 데이터 분할 (누수 방지 : route 그룹 분할)

```
SEED = 42
TEST_SIZE = 0.2

# 1) 그룹 정보(route) 준비
groups = df_cleaned.loc[X.index, "route"]

# 2) 분할
# GroupShuffleSplit으로 무작위가 아니라 route 단위로 분할해 누수 방지
gss = GroupShuffleSplit(n_splits=1, test_size=TEST_SIZE, random_state=SEED)
tr_idx, te_idx = next(gss.split(X, y, groups=groups))
X_train, X_test = X.iloc[tr_idx], X.iloc[te_idx]
X_linear_train, X_linear_test = X_linear.iloc[tr_idx], X_linear.iloc[te_idx]
y_train, y_test = y.iloc[tr_idx], y.iloc[te_idx]
g_train, g_test = groups.iloc[tr_idx], groups.iloc[te_idx]
# 누수 체크: train/test 간 route 겹침 없어야 함
overlap = set(g_train.unique()) & set(g_test.unique())
print(f"Route overlap: {len(overlap)}")
assert len(overlap) == 0, f"경고: train/test에 동일 route가 있습니다: {list(overlap)[:5]}"

print("- X_train / X_test:", X_train.shape, X_test.shape)
print("- X_linear_train / X_linear_test:", X_linear_train.shape, X_linear_test.shape)
print("- y_train / y_test:", y_train.shape, y_test.shape)

Route overlap: 0
- X_train / X_test: (242201, 37) (54976, 37)
- X_linear_train / X_linear_test: (242201, 31) (54976, 31)
- y_train / y_test: (242201, 2) (54976, 2)
```

```

# 숫자형 X1, y1 준비
num_features = [col for col in df_encoded.columns
                 if col not in targets and df_encoded[col].dtype in [np.int64, np.float64, bool]]
num_features_linear = [col for col in df_encoded_linear.columns
                        if col not in targets and df_encoded_linear[col].dtype in [np.int64, np.float64, bool]]

X1 = df_encoded[num_features]
X1_linear = df_encoded_linear[num_features_linear]
y1 = df_encoded[targets]

# 앞에서 얻은 train/test 인덱스 재사용
X1_train, X1_test = X1.iloc[tr_idx], X1.iloc[te_idx]
X1_linear_train, X1_linear_test = X1_linear.iloc[tr_idx], X1_linear.iloc[te_idx]
y1_train, y1_test = y1.iloc[tr_idx], y1.iloc[te_idx]

print(" 기존 분할 재사용 완료")
print(" - X1_train:", X1_train.shape, "X1_test:", X1_test.shape)
print(" - X1_linear_train:", X1_linear_train.shape, "X1_linear_test:", X1_linear_test.shape)
print(" - y1_train:", y1_train.shape, "y1_test:", y1_test.shape)

```

기존 분할 재사용 완료

- X1\_train: (242201, 37) X1\_test: (54976, 37)
- X1\_linear\_train: (242201, 31) X1\_linear\_test: (54976, 31)
- y1\_train: (242201, 2) y1\_test: (54976, 2)

# # Ridge / Lasso

## # 스케일링

```
scaler = StandardScaler()
X1_linear_train = scaler.fit_transform(X1_linear_train)
X1_linear_test = scaler.transform(X1_linear_test)
```

```
alphas = np.logspace(-2, 2, 20)
```

## # 학습 함수 정의

```
def train_ridge_lasso(X, y, alphas, model_type):
    Model = Ridge if model_type == "ridge" else Lasso
    model = Model(random_state=42, max_iter=5000)
    grid = GridSearchCV(model, {"alpha": alphas}, cv=5, scoring="r2")
    grid.fit(X, y)
    best_model = grid.best_estimator_
    best_alpha = grid.best_params_["alpha"]
    return best_model, best_alpha
```

```
y_price = y1_train["price_log"]
y_dur = y1_train["duration_log"]
```

## # Ridge

```
ridge_price, alpha_ridge_price = train_ridge_lasso(X1_linear_train, y_price, alphas, model_type="ridge")
ridge_dur, alpha_ridge_dur = train_ridge_lasso(X1_linear_train, y_dur, alphas, model_type="ridge")
```

## # Lasso

```
lasso_price, alpha_lasso_price = train_ridge_lasso(X1_linear_train, y_price, alphas, model_type="lasso")
lasso_dur, alpha_lasso_dur = train_ridge_lasso(X1_linear_train, y_dur, alphas, model_type="lasso")
```

## # 성능 평가

```
# =====
```

```
def print_model_scores(model, X_train, X_test, y_train, y_test):
    print(f"R2 train: {model.score(X_train, y_train):.3f} | R2 test: {model.score(X_test, y_test):.3f}")
```

```
print("=== Ridge/Lasso (log(price)) ===")
print(f" Ridge best alpha: {alpha_ridge_price}")
print_model_scores(ridge_price, X1_linear_train, X1_linear_test, y1_train["price_log"], y1_test["price_log"])
```

```
print(f" Lasso best alpha: {alpha_lasso_price}")
print_model_scores(lasso_price, X1_linear_train, X1_linear_test, y1_train["price_log"], y1_test["price_log"])
```

```
print("=== Ridge/Lasso (log(duration)) ===")
print(f" Ridge best alpha: {alpha_ridge_dur}")
print_model_scores(ridge_dur, X1_linear_train, X1_linear_test, y1_train["duration_log"], y1_test["duration_log"])
```

```
print(f" Lasso best alpha: {alpha_lasso_dur}")
print_model_scores(lasso_dur, X1_linear_train, X1_linear_test, y1_train["duration_log"], y1_test["duration_log"])
```

```
=== Ridge/Lasso (log(price)) ===
Ridge best alpha: 0.01
R2 train: 0.915 | R2 test: 0.909
Lasso best alpha: 0.01
R2 train: 0.910 | R2 test: 0.904
=== Ridge/Lasso (log(duration)) ===
Ridge best alpha: 100.0
R2 train: 0.594 | R2 test: 0.653
Lasso best alpha: 0.01
R2 train: 0.588 | R2 test: 0.644
```

## # 변수 중요도 (Ridge / Lasso)

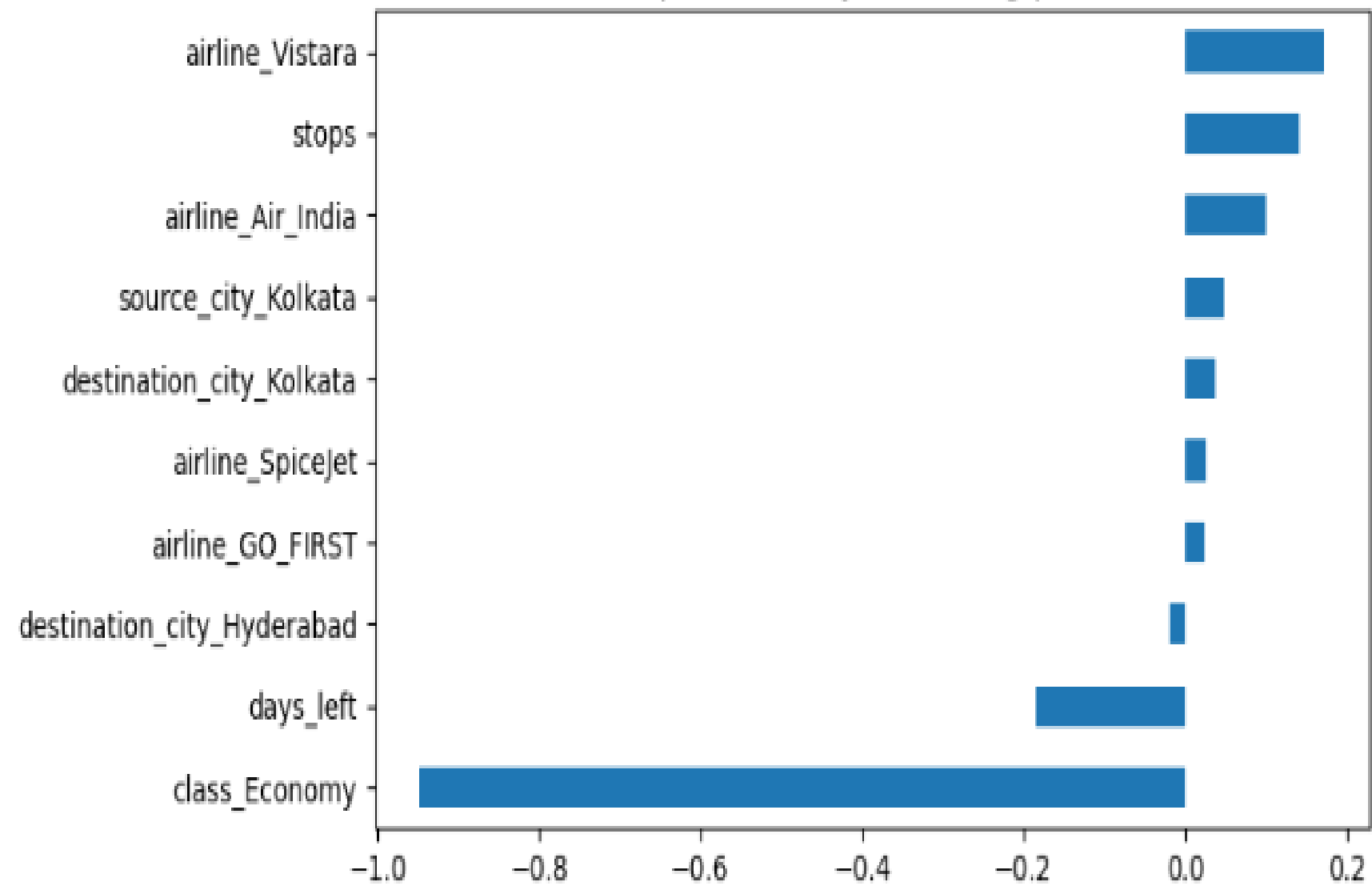
```
# =====  
# 변수 중요도 (Lasso)  
# =====  
coef_price = pd.Series(lasso_price.coef_, index=X1_linear.columns).sort_values(key=np.abs, ascending=False)  
coef_dur = pd.Series(lasso_dur.coef_, index=X1_linear.columns).sort_values(key=np.abs, ascending=False)  
  
print("\nTop 변수 (log(price)):")  
print(coef_price.head(10))  
print("\nTop 변수 (log(duration)):")  
print(coef_dur.head(10))  
  
# =====  
# 변수 중요도 시각화  
# =====  
def plot_top_features(coef_series, title):  
    plt.figure(figsize=(8, 4))  
    coef_series.head(10).sort_values().plot(kind='barh')  
    plt.title(title)  
    plt.tight_layout()  
    plt.show()  
  
print("\n")  
plot_top_features(coef_price, "Top Features by Lasso - log(price)")  
print("\n")  
plot_top_features(coef_dur, "Top Features by Lasso - log(duration)")
```

```
Top 변수 (log(price)):  
class_Economy      -0.946832  
days_left         -0.184345  
airline_Vistara      0.172888  
stops              0.142650  
airline_Air_India    0.100089  
source_city_Kolkata  0.048918  
destination_city_Kolkata 0.036540  
airline_SpiceJet     0.025681  
airline_GO_FIRST     0.024673  
destination_city_Hyderabad -0.019531  
dtype: float64
```

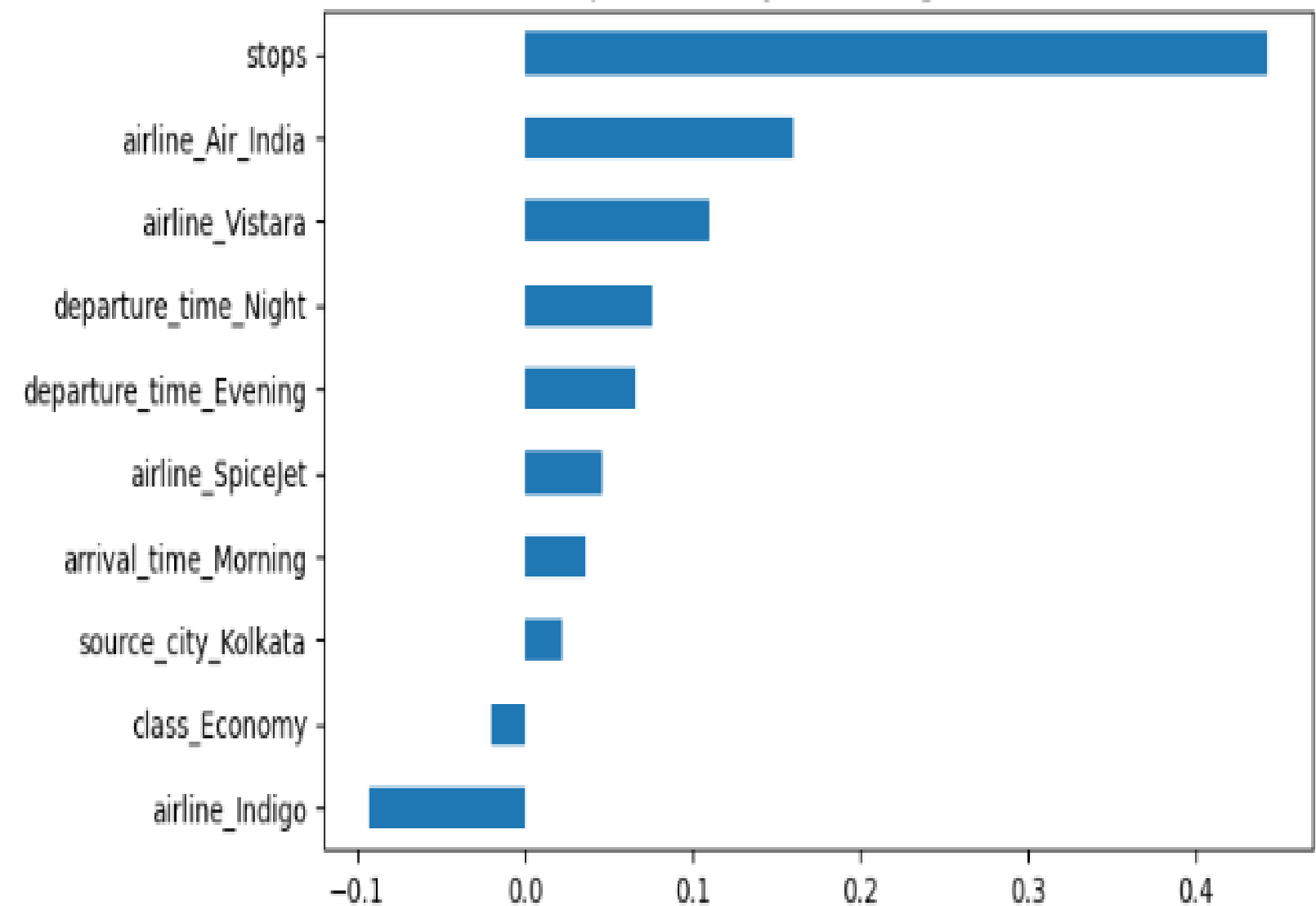
```
Top 변수 (log(duration)):  
stops              0.443272  
airline_Air_India  0.160163  
airline_Vistara    0.109821  
airline_Indigo     -0.093127  
departure_time_Night 0.076342  
departure_time_Evening 0.066423  
airline_SpiceJet    0.046044  
arrival_time_Morning 0.036521  
source_city_Kolkata 0.022051  
class_Economy      -0.019731  
dtype: float64
```

## # 변수 중요도 (Ridge / Lasso)

Top Features by Lasso - log(price)



Top Features by Lasso - log(duration)



# # Random Forest

```
rf_price = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
rf_dur = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
```

# 학습

```
rf_price.fit(X1_train, y1_train["price_log"])
rf_dur.fit(X1_train, y1_train["duration_log"])
```

# 예측

```
y_train_pred_price = rf_price.predict(X1_train)
y_test_pred_price = rf_price.predict(X1_test)
y_train_pred_dur = rf_dur.predict(X1_train)
y_test_pred_dur = rf_dur.predict(X1_test)
```

# 성능 평가

```
def evaluate_model(y_train, y_test, y_train_pred, y_test_pred, label):
    mse = mean_squared_error(y_test, y_test_pred)
    rmse = np.sqrt(mse)

    print(f"=== {label} ===")
    print(f"R2 train: {r2_score(y_train, y_train_pred):.3f}")
    print(f"R2 test: {r2_score(y_test, y_test_pred):.3f}")
    print(f"RMSE test: {rmse:.3f}")
    print(f"exp(RMSE): {np.exp(rmse):.3f}")
    print()
```

```
evaluate_model(
    y1_train["price_log"], y1_test["price_log"],
    y_train_pred_price, y_test_pred_price,
    "Random Forest (log(price))"
)
```

```
evaluate_model(
    y1_train["duration_log"], y1_test["duration_log"],
    y_train_pred_dur, y_test_pred_dur,
    "Random Forest (log(duration))"
)
```

```
=== Random Forest (log(price)) ===
R2 train: 0.987
R2 test: 0.947
RMSE test: 0.258
exp(RMSE): 1.295
```

```
=== Random Forest (log(duration)) ===
R2 train: 0.962
R2 test: 0.879
RMSE test: 0.268
exp(RMSE): 1.308
```

## # 변수 중요도 (Random Forest)

```
importances_price = pd.Series(rf_price.feature_importances_, index=X1.columns).sort_values(ascending=False)
importances_dur = pd.Series(rf_dur.feature_importances_, index=X1.columns).sort_values(ascending=False)

print("Top Features (log(price)) - Random Forest:")
print(importances_price.head(10))

print("\nTop Features (log(duration())) - Random Forest:")
print(importances_dur.head(10))

# =====
# 변수 중요도 시각화
# =====
def plot_importance(importances, title):
    plt.figure(figsize=(8, 4))
    importances.head(10).sort_values().plot(kind='barh')
    plt.title(title)
    plt.tight_layout()
    plt.show()

print("\n")
plot_importance(importances_price, "Top Features by RF - log(price)")
print("\n")
plot_importance(importances_dur, "Top Features by RF - log(duration)")
```

Top Features (log(price)) - Random Forest:

|                     |          |
|---------------------|----------|
| class_Business      | 0.596642 |
| class_Economy       | 0.243655 |
| days_left           | 0.069654 |
| stops               | 0.029686 |
| route_hhi           | 0.014293 |
| airline_AirAsia     | 0.009147 |
| airline_Indigo      | 0.005567 |
| airline_Air_India   | 0.004928 |
| airline_Vistara     | 0.003283 |
| source_city_Kolkata | 0.002702 |

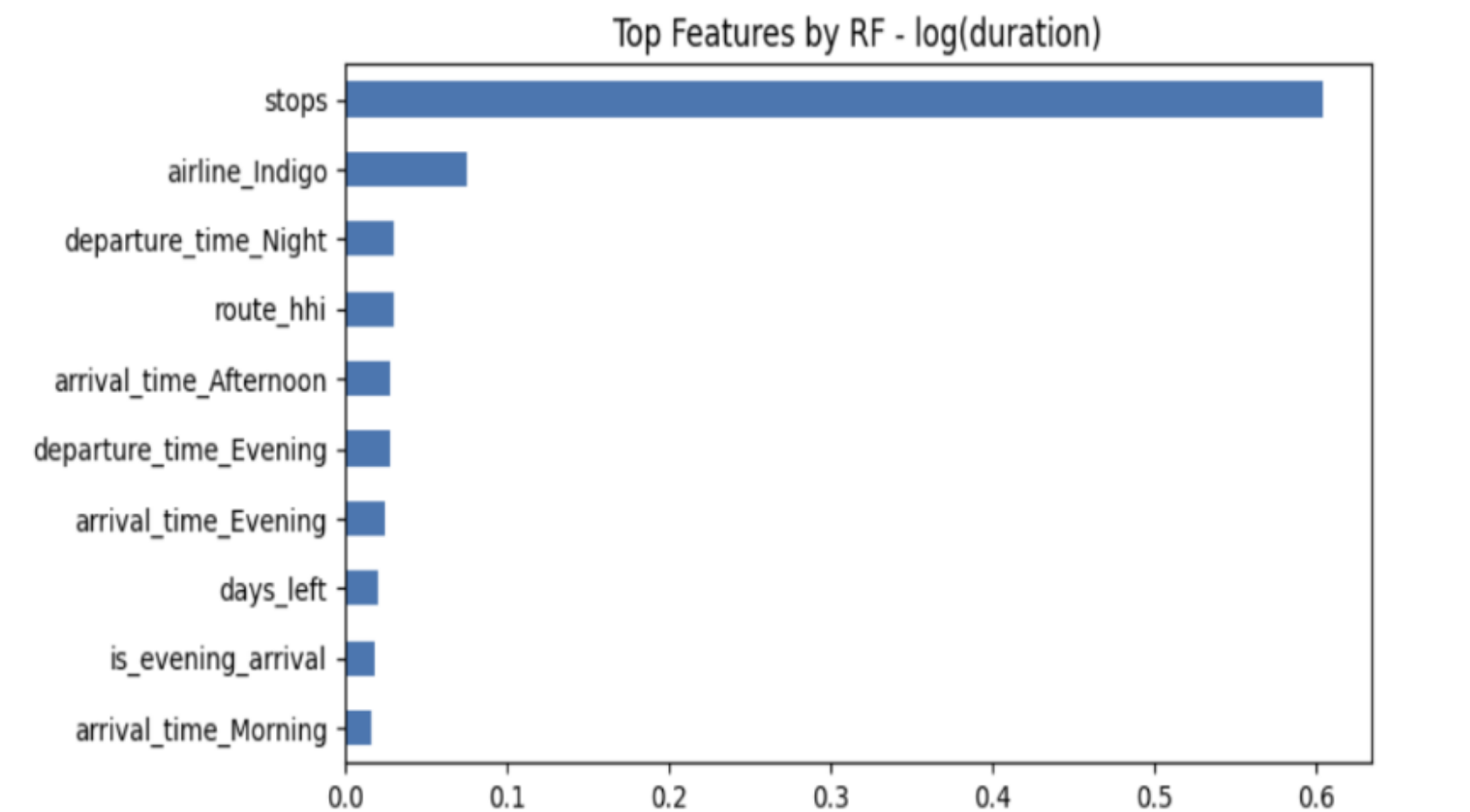
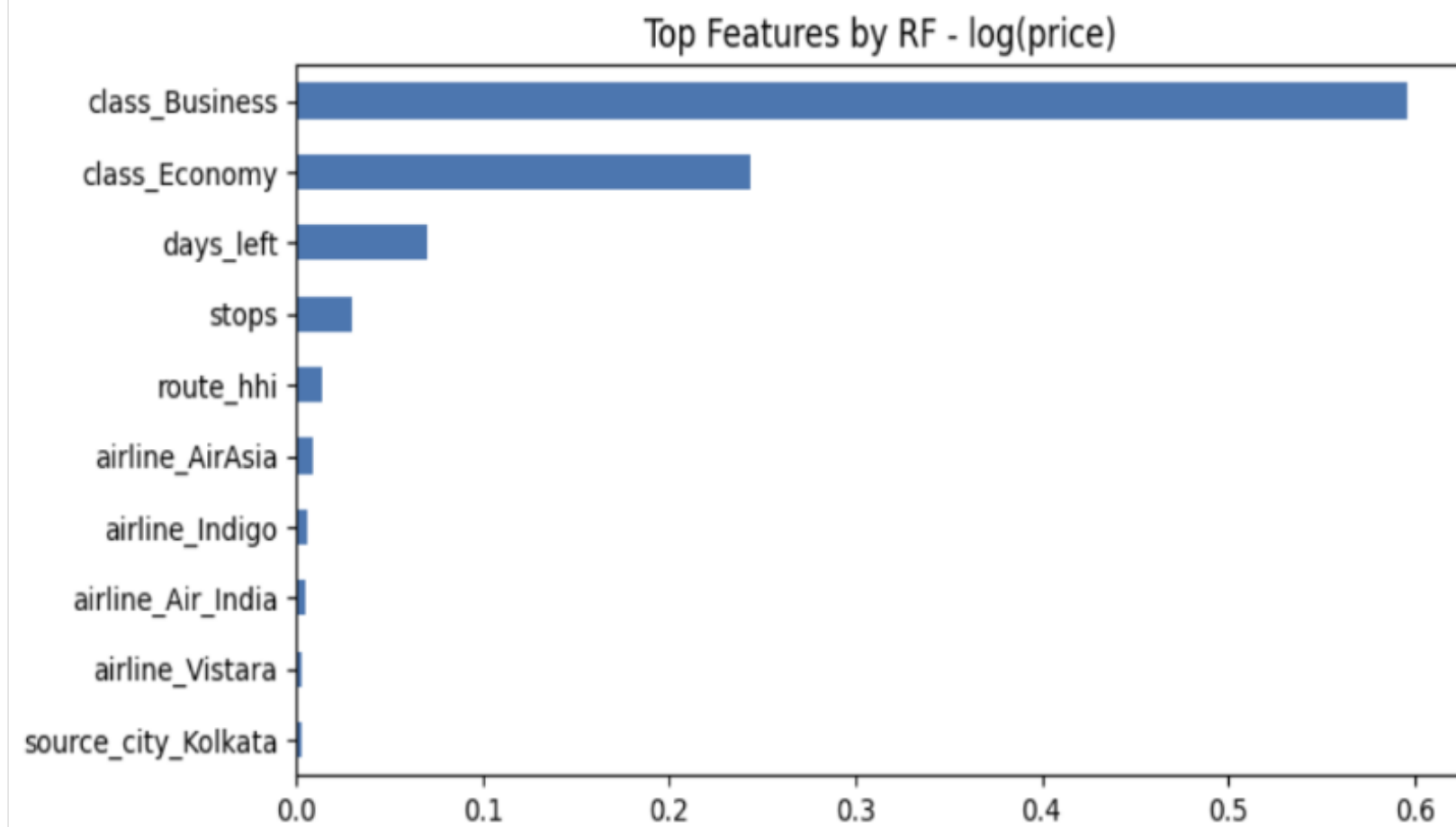
dtype: float64

Top Features (log(duration())) - Random Forest:

|                        |          |
|------------------------|----------|
| stops                  | 0.603997 |
| airline_Indigo         | 0.075626 |
| departure_time_Night   | 0.030293 |
| route_hhi              | 0.030066 |
| arrival_time_Afternoon | 0.028759 |
| departure_time_Evening | 0.028001 |
| arrival_time_Evening   | 0.025128 |
| days_left              | 0.020879 |
| is_evening_arrival     | 0.018584 |
| arrival_time_Morning   | 0.015974 |

dtype: float64

## # 변수 중요도 (Random Forest)



# # XGBoost

```
xgb_price = XGBRegressor(n_estimators=200, learning_rate=0.1, max_depth=6, random_state=42, n_jobs=-1)
xgb_dur = XGBRegressor(n_estimators=200, learning_rate=0.1, max_depth=6, random_state=42, n_jobs=-1)
```

# 학습

```
xgb_price.fit(X1_train, y1_train["price_log"])
xgb_dur.fit(X1_train, y1_train["duration_log"])
```

# 예측

```
y_train_pred_price_xgb = xgb_price.predict(X1_train)
y_test_pred_price_xgb = xgb_price.predict(X1_test)
y_train_pred_dur_xgb = xgb_dur.predict(X1_train)
y_test_pred_dur_xgb = xgb_dur.predict(X1_test)
```

# 성능 평가

```
evaluate_model(
    y1_train["price_log"], y1_test["price_log"],
    y_train_pred_price_xgb, y_test_pred_price_xgb,
    "XGBoost (log(price))"
)

evaluate_model(
    y1_train["duration_log"], y1_test["duration_log"],
    y_train_pred_dur_xgb, y_test_pred_dur_xgb,
    "XGBoost (log(duration))"
)
```

=== XGBoost (log(price)) ===

$R^2$  train: 0.972

$R^2$  test: 0.955

RMSE test: 0.238

exp(RMSE): 1.268

=== XGBoost (log(duration)) ===

$R^2$  train: 0.932

$R^2$  test: 0.884

RMSE test: 0.262

exp(RMSE): 1.300

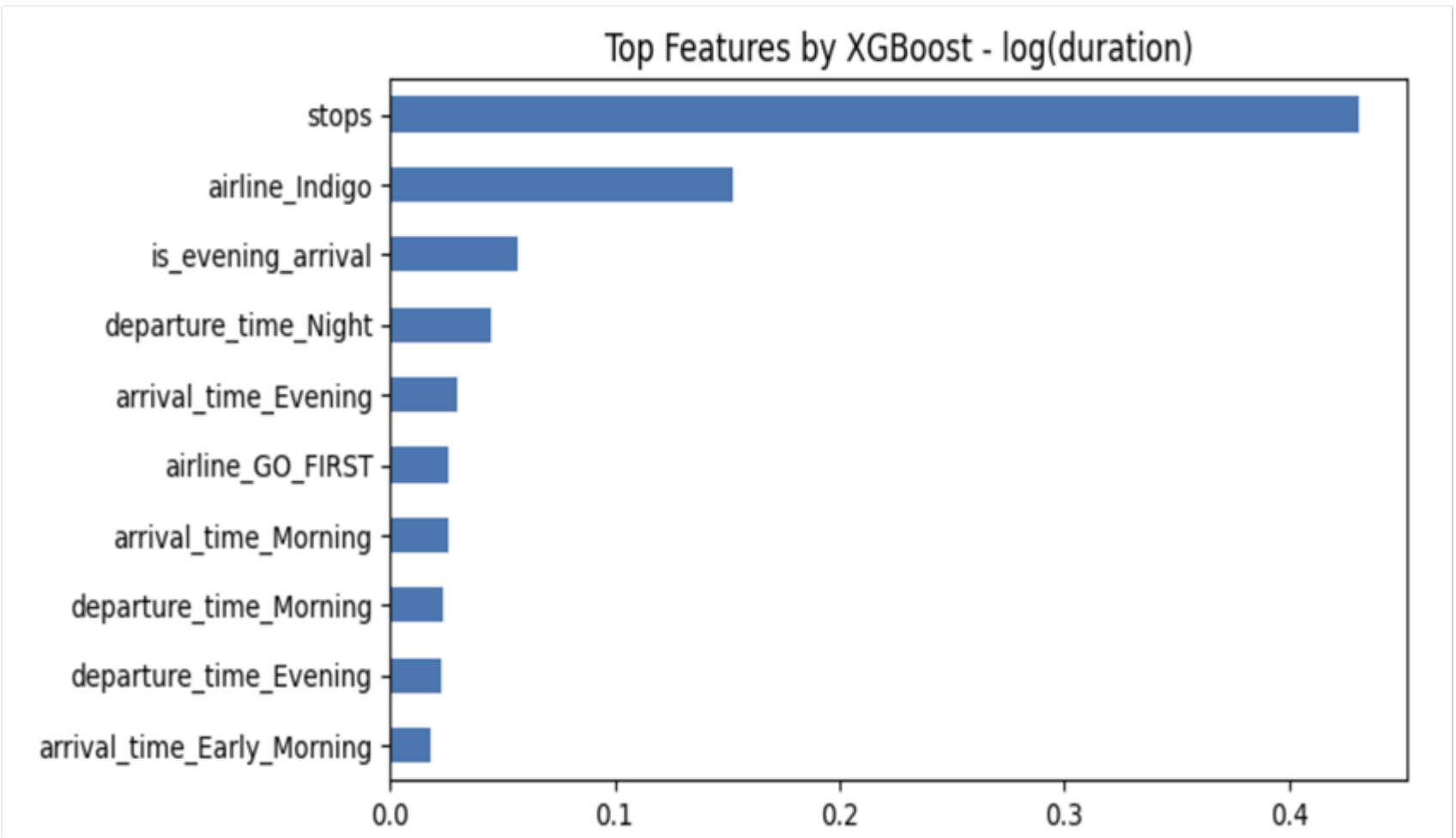
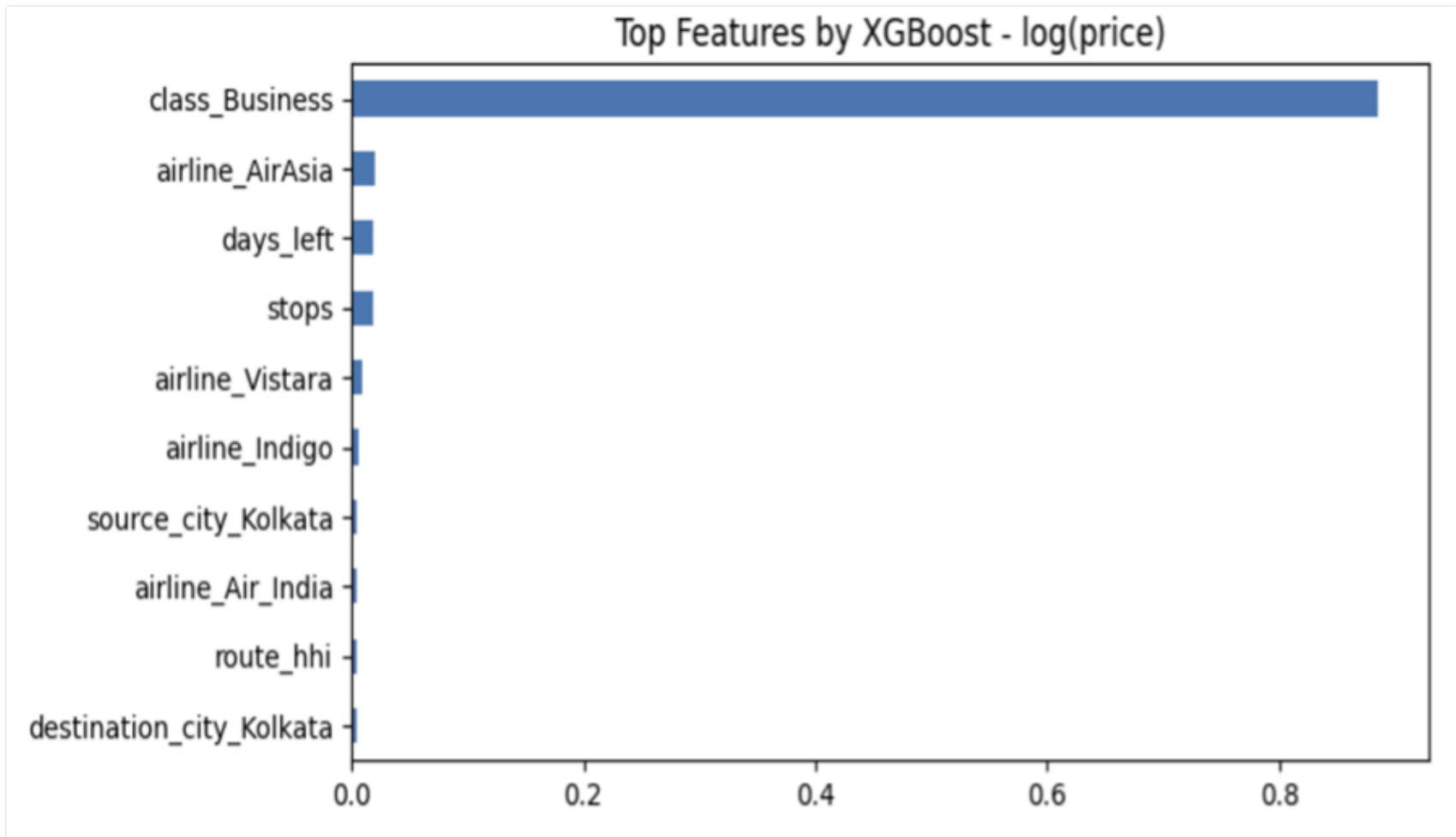
## # 변수 중요도 (XGBoost)

```
# =====  
# 변수 중요도 (XGBoost)  
# =====  
importances_price_xgb = pd.Series(xgb_price.feature_importances_, index=X1.columns).sort_values(ascending=False)  
importances_dur_xgb = pd.Series(xgb_dur.feature_importances_, index=X1.columns).sort_values(ascending=False)  
  
print("Top Features (log(price)) - XGBoost:")  
print(importances_price_xgb.head(10))  
  
print("\nTop Features (duration) - XGBoost:")  
print(importances_dur_xgb.head(10))  
  
# =====  
# 변수 중요도 시각화  
# =====  
print("\n")  
plot_importance(importances_price_xgb, "Top Features by XGBoost - log(price)")  
print("\n")  
plot_importance(importances_dur_xgb, "Top Features by XGBoost - duration")
```

```
Top Features (log(price)) - XGBoost:  
class_Business      0.884769  
airline_AirAsia     0.020280  
days_left          0.018991  
stops               0.018395  
airline_Vistara     0.009701  
airline_Indigo      0.006716  
source_city_Kolkata 0.005306  
airline_Air_India   0.004776  
route_hhi           0.004389  
destination_city_Kolkata 0.004055  
dtype: float32
```

```
Top Features (log(duration)) - XGBoost:  
stops               0.430735  
airline_Indigo      0.152550  
is_evening_arrival  0.057182  
departure_time_Night 0.044640  
arrival_time_Evening 0.030247  
airline_G0_FIRST    0.026194  
arrival_time_Morning 0.026139  
departure_time_Morning 0.023591  
departure_time_Evening 0.022925  
arrival_time_Early_Morning 0.018032  
dtype: float32
```

# # 변수 중요도 (XGBoost)



# # OLS (가격)

```
X_float = X1_train.astype(float)
X_const = sm.add_constant(X_float)
ols_price = sm.OLS(y1_train["price_log"], X_const).fit()
ols_dur = sm.OLS(y1_train["duration_log"], X_const).fit()

print("\n=== OLS 결과 (log(price)) ===")
print(ols_price.summary())

print("\n=== OLS 결과 (log(duration)) ===")
print(ols_dur.summary())
```

=== OLS 결과 (log(price)) ===

| OLS Regression Results |                  |                     |           |
|------------------------|------------------|---------------------|-----------|
| =====                  |                  |                     |           |
| Dep. Variable:         | price_log        | R-squared:          | 0.915     |
| Model:                 | OLS              | Adj. R-squared:     | 0.915     |
| Method:                | Least Squares    | F-statistic:        | 9.035e+04 |
| Date:                  | Sat, 27 Sep 2025 | Prob (F-statistic): | 0.00      |
| Time:                  | 04:11:11         | Log-Likelihood:     | -70883.   |
| No. Observations:      | 242201           | AIC:                | 1.418e+05 |
| Df Residuals:          | 242171           | BIC:                | 1.421e+05 |
| Df Model:              | 29               |                     |           |
| Covariance Type:       | nonrobust        |                     |           |

|                              | coef    |
|------------------------------|---------|
| -----                        |         |
| const                        | 3.9747  |
| stops                        | 0.3966  |
| days_left                    | -0.0141 |
| airline_AirAsia              | 0.2447  |
| airline_Air_India            | 0.8008  |
| airline_GO_FIRST             | 0.6766  |
| airline_Indigo               | 0.5807  |
| airline_SpiceJet             | 0.7564  |
| airline_Vistara              | 0.9155  |
| source_city_Bangalore        | 0.6786  |
| source_city_Chennai          | 0.6168  |
| source_city_Delhi            | 0.6355  |
| source_city_Hyderabad        | 0.5975  |
| source_city_Kolkata          | 0.8039  |
| source_city_Mumbai           | 0.6423  |
| destination_city_Bangalore   | 0.6835  |
| destination_city_Chennai     | 0.6366  |
| destination_city_Delhi       | 0.6568  |
| destination_city_Hyderabad   | 0.5811  |
| destination_city_Kolkata     | 0.7719  |
| destination_city_Mumbai      | 0.6447  |
| departure_time_Afternoon     | 0.8337  |
| departure_time_Early_Morning | 0.2828  |
| departure_time_Evening       | 0.8258  |
| departure_time_Late_Night    | 0.8990  |
| departure_time_Morning       | 0.8585  |
| departure_time_Night         | 0.2749  |
| arrival_time_Afternoon       | 0.8674  |
| arrival_time_Early_Morning   | 0.8023  |
| arrival_time_Evening         | 0.3016  |
| arrival_time_Late_Night      | 0.8617  |
| arrival_time_Morning         | 0.8502  |
| arrival_time_Night           | 0.2915  |
| class_Business               | 3.0122  |
| class_Economy                | 0.9625  |
| is_redeye_from_dep           | 0.5577  |
| is_evening_arrival           | 0.5931  |
| route_hhi                    | -0.2114 |

## # OLS (시간)

```
X_float = X1_train.astype(float)
X_const = sm.add_constant(X_float)
ols_price = sm.OLS(y1_train["price_log"], X_const).fit()
ols_dur = sm.OLS(y1_train["duration_log"], X_const).fit()

print("\n=== OLS 결과 (log(price)) ===")
print(ols_price.summary())

print("\n=== OLS 결과 (log(duration)) ===")
print(ols_dur.summary())
```

=== OLS 결과 (log(duration)) ===

### OLS Regression Results

```
=====
Dep. Variable:      duration_log    R-squared:      0.594
Model:              OLS            Adj. R-squared: 0.594
Method:             Least Squares   F-statistic:   1.224e+04
Date:               Sat, 27 Sep 2025 Prob (F-statistic): 0.00
Time:               04:11:11        Log-Likelihood: -1.5477e+05
No. Observations:   242201          AIC:           3.096e+05
Df Residuals:       242171          BIC:           3.099e+05
Df Model:           29
Covariance Type:    nonrobust
```

|                              | coef    |
|------------------------------|---------|
| const                        | 0.4953  |
| stops                        | 1.1415  |
| days_left                    | -0.0005 |
| airline_AirAsia              | -0.1542 |
| airline_Air_India            | 0.3759  |
| airline_GO_FIRST             | 0.0179  |
| airline_Indigo               | -0.2859 |
| airline_SpiceJet             | 0.3144  |
| airline_Vistara              | 0.2272  |
| source_city_Bangalore        | 0.0715  |
| source_city_Chennai          | 0.0820  |
| source_city_Delhi            | 0.0370  |
| source_city_Hyderabad        | 0.0980  |
| source_city_Kolkata          | 0.1675  |
| source_city_Mumbai           | 0.0393  |
| destination_city_Bangalore   | 0.0760  |
| destination_city_Chennai     | 0.1033  |
| destination_city_Delhi       | 0.0509  |
| destination_city_Hyderabad   | 0.1332  |
| destination_city_Kolkata     | 0.1187  |
| destination_city_Mumbai      | 0.0132  |
| departure_time_Afternoon     | 0.0157  |
| departure_time_Early_Morning | -0.0521 |
| departure_time_Evening       | 0.2254  |
| departure_time_Late_Night    | 0.0965  |
| departure_time_Morning       | 0.0384  |
| departure_time_Night         | 0.1715  |
| arrival_time_Afternoon       | 0.0843  |
| arrival_time_Early_Morning   | 0.1701  |
| arrival_time_Evening         | 0.0440  |
| arrival_time_Late_Night      | -0.0075 |
| arrival_time_Morning         | 0.1930  |
| arrival_time_Night           | 0.0115  |
| class_Business               | 0.2713  |
| class_Economy                | 0.2240  |
| is_redeye_from_dep           | 0.1194  |
| is_evening_arrival           | 0.0555  |
| route_hhi                    | -0.1445 |

## # 시간/가격 동시 예측 모델 (Random Forest)

### # 랜덤포레스트

```
base_model = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
multi_model_rf = MultiOutputRegressor(base_model)
```

### # 학습

```
multi_model_rf.fit(X1_train, y1_train)
```

### # 예측

```
y1_train_multi_rf = multi_model_rf.predict(X1_train)
y1_test_multi_rf = multi_model_rf.predict(X1_test)
```

### # 성능 평가

```
def evaluate_multi_output(y_train, y_train_pred, y_test, y_test_pred, target_names):
    print("=== MultiOutput Regression Evaluation ===")
    for i, name in enumerate(target_names):
        r2_train = r2_score(y_train.iloc[:, i], y_train_pred[:, i])
        rmse_train = np.sqrt(mean_squared_error(y_train.iloc[:, i], y_train_pred[:, i]))
        r2_test = r2_score(y_test.iloc[:, i], y_test_pred[:, i])
        rmse_test = np.sqrt(mean_squared_error(y_test.iloc[:, i], y_test_pred[:, i]))

        print(f"\n[{name}]")
        print(f" R² train: {r2_train:.3f} | RMSE train: {rmse_train:.3f}")
        print(f" exp(RMSE train): {np.exp(rmse_train):.3f}")
        print(f" R² test: {r2_test:.3f} | RMSE test: {rmse_test:.3f}")
        print(f" exp(RMSE test): {np.exp(rmse_test):.3f}")

evaluate_multi_output(y1_train, y1_train_multi_rf, y1_test, y1_test_multi_rf, y1_train.columns)
```

```
=== MultiOutput Regression Evaluation ===
```

```
[price_log]
```

```
R² train: 0.987 | RMSE train: 0.129
exp(RMSE train): 1.138
R² test: 0.947 | RMSE test: 0.258
exp(RMSE test): 1.295
```

```
[duration_log]
```

```
R² train: 0.962 | RMSE train: 0.140
exp(RMSE train): 1.150
R² test: 0.879 | RMSE test: 0.268
exp(RMSE test): 1.308
```

## # 시간/가격 동시 예측 모델 (XGBoost)

```
# XGBoost
xgb_base = XGBRegressor(n_estimators=200, max_depth=6, learning_rate=0.1, random_state=42, n_jobs=-1)
multi_model_xgb = MultiOutputRegressor(xgb_base)

# 학습
multi_model_xgb.fit(X1_train, y1_train)

# 예측
y1_train_multi_xgb = multi_model_xgb.predict(X1_train)
y1_test_multi_xgb = multi_model_xgb.predict(X1_test)
```

### # 성능 평가

```
evaluate_multi_output(y1_train, y1_train_multi_xgb, y1_test, y1_test_multi_xgb, y1_train.columns)
```

=== MultiOutput Regression Evaluation ===

[price\_log]

R<sup>2</sup> train: 0.972 | RMSE train: 0.186

exp(RMSE train): 1.204

R<sup>2</sup> test: 0.955 | RMSE test: 0.238

exp(RMSE test): 1.268

[duration\_log]

R<sup>2</sup> train: 0.932 | RMSE train: 0.188

exp(RMSE train): 1.206

R<sup>2</sup> test: 0.884 | RMSE test: 0.262

exp(RMSE test): 1.300

```
# KFold
cv = KFold(n_splits=5, shuffle=True, random_state=42)
r2_scores_multi = cross_val_score(multi_model_xgb, X1_train, y1_train, cv=cv, scoring="r2")
print("Cross-Validation (Multi-Output) R² :", r2_scores_multi)
print(" - 평균 R² :", np.mean(r2_scores_multi))
```

---

Cross-Validation (Multi-Output) R² : [0.88840622 0.88835621 0.89131808 0.88397503 0.88926768]  
- 평균 R² : 0.888264641459656

## # 하이퍼파라미터 튜닝

```
base_model = XGBRegressor(random_state=42, n_jobs=-1)
multi_model = MultiOutputRegressor(base_model)

# 파라미터 설정
param_grid = {
    "estimator__n_estimators": [100, 200],
    "estimator__learning_rate": [0.05, 0.1],
    "estimator__max_depth": [4, 6],
    "estimator__subsample": [0.7, 1.0],
}

# GridSearchCV 설정
cv = KFold(n_splits=3, shuffle=True, random_state=42)
search = GridSearchCV(multi_model, param_grid, cv=cv, scoring="r2", verbose=1)

# 학습
search.fit(X1_train, y1_train)

# 결과 출력
print("Best Params:", search.best_params_)
print("Best Score:", search.best_score_)
```

Fitting 3 folds for each of 16 candidates, totalling 48 fits

Best Params: {'estimator\_\_learning\_rate': 0.1, 'estimator\_\_max\_depth': 6, 'estimator\_\_n\_estimators': 200, 'estimator\_\_subsample': 0.7}

Best Score: 0.9516134262084961

## # 최적 모델 학습

```
# 최적 모델 꺼내기
best_model = search.best_estimator_

# 전체 학습 데이터로 다시 fit
best_model.fit(X1_train, y1_train)

# 예측
y_train_pred = best_model.predict(X1_train)
y_test_pred = best_model.predict(X1_test)

targets = ["price_log", "duration_log"]

print("=== 최적 XGBoost 모델 평가 ===\n")

for i, target in enumerate(targets):
    print(f"[{target}]")
    r2_train = r2_score(y1_train[target], y_train_pred[:, i])
    r2_test = r2_score(y1_test[target], y_test_pred[:, i])
    rmse_train = np.sqrt(mean_squared_error(y1_train[target], y_train_pred[:, i]))
    rmse_test = np.sqrt(mean_squared_error(y1_test[target], y_test_pred[:, i]))

    print(f"R² train: {r2_train:.3f} | RMSE train: {rmse_train:.3f}")
    print(f"exp(RMSE train): {np.exp(rmse_train):.3f}")
    print(f"R² test: {r2_test:.3f} | RMSE test: {rmse_test:.3f}")
    print(f"exp(RMSE test): {np.exp(rmse_test):.3f}")

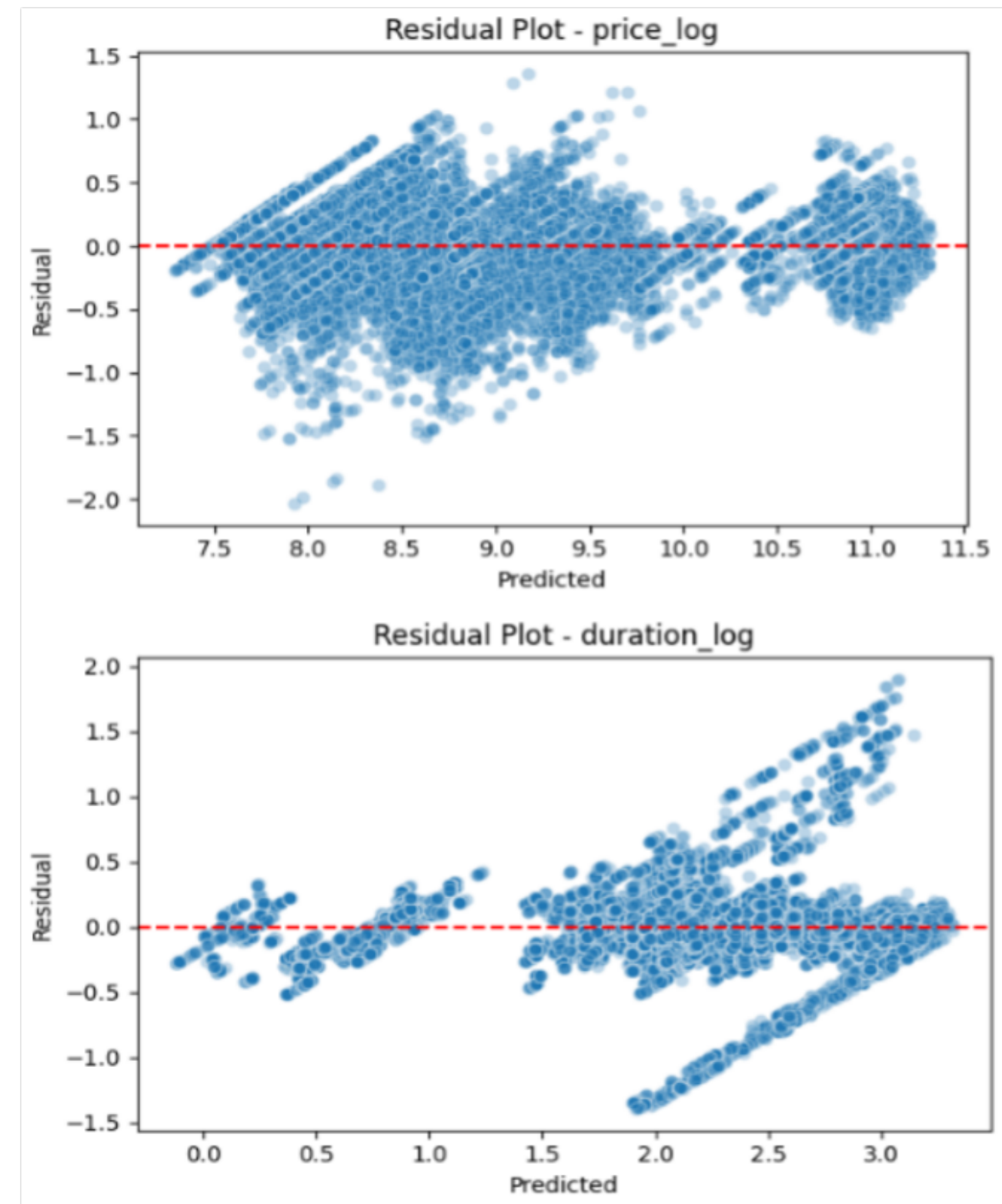
=== 최적 XGBoost 모델 평가 ===

[price_log]
R² train: 0.973 | RMSE train: 0.184
exp(RMSE train): 1.203
R² test: 0.955 | RMSE test: 0.237
exp(RMSE test): 1.268
[duration_log]
R² train: 0.935 | RMSE train: 0.184
exp(RMSE train): 1.202
R² test: 0.886 | RMSE test: 0.260
exp(RMSE test): 1.296
```

## # 잔차 시각화

```
residuals = y_test_pred - y1_test.values
targets = ["price_log", "duration_log"]

for i, target in enumerate(targets):
    plt.figure(figsize=(6, 4))
    sns.scatterplot(x=y_test_pred[:, i], y=residuals[:, i], alpha=0.3)
    plt.axhline(0, color='red', linestyle='--')
    plt.xlabel("Predicted")
    plt.ylabel("Residual")
    plt.title(f"Residual Plot - {target}")
    plt.tight_layout()
    plt.show()
```



## # 항공사 추천

```
def recommend_airline(user_input, model, top_k=3, time_weight=0.5):
    # 추천 후보 만들기
    candidates = []
    for airline in df['airline'].unique():
        row = user_input.copy()
        row["airline"] = airline

        # 자동 파생 변수 계산
        row["is_redeye_from_dep"] = int(row["departure_time"] in ["Night", "Early_Morning"])
        row["is_evening_arrival"] = int(row["arrival_time"] in ["Evening", "Night"])

        route = row["source_city"] + "-" + row["destination_city"]

        # HHI 계산
        hhi_row = route_hhi[route_hhi["route"] == route]
        row["route_hhi"] = float(hhi_row["route_hhi"].values[0]) if not hhi_row.empty else 0.5 # 기본값 0.5

        candidates.append(row)

    # 데이터프레임으로 만들고 인코딩
    df_cand = pd.DataFrame(candidates)
    df_cand_encoded = pd.get_dummies(df_cand)

    # 모델에 들어갈 컬럼 정렬 및 누락 컬럼 처리
    for col in X1_train.columns:
        if col not in df_cand_encoded.columns:
            df_cand_encoded[col] = 0
    df_cand_encoded = df_cand_encoded[X1_train.columns]

    # 예측
    preds = model.predict(df_cand_encoded)
    df_cand["pred_price_log"] = preds[:, 0]
    df_cand["pred_price"] = np.exp(preds[:, 0])
    df_cand["pred_duration_log"] = preds[:, 1]
    df_cand["pred_duration"] = np.exp(preds[:, 1])

    # 정규화
    price_norm = (df_cand["pred_price"] - df_cand["pred_price"].min()) / (df_cand["pred_price"].max() - df_cand["pred_price"].min())
    dur_norm = (df_cand["pred_duration"] - df_cand["pred_duration"].min()) / (df_cand["pred_duration"].max() - df_cand["pred_duration"].min())

    # 점수 계산
    df_cand["score"] = time_weight * dur_norm + (1 - time_weight) * price_norm

    # 결과 정렬 및 반환
    df_result = df_cand.sort_values("score").reset_index(drop=True).head(top_k)
    return df_result[["airline", "pred_price", "pred_duration", "score"]]
```

## 결론 도출

```
user_input = {  
    "source_city": "Delhi",  
    "destination_city": "Mumbai",  
    "class": "Economy",  
    "days_left": 10,  
    "stops": 0,  
    "departure_time": "Morning",  
    "arrival_time": "Evening"  
}
```

```
result = recommend_airline(user_input, best_model, time_weight=0.7)  
print(result)  
print()
```

```
print("==== 최적 항공사 =====")  
best_airline = result['airline'].iloc[0]  
print(best_airline)
```

|   | airline   | pred_price  | pred_duration | score    |
|---|-----------|-------------|---------------|----------|
| 0 | Air_India | 7246.861816 | 1.968869      | 0.052672 |
| 1 | Vistara   | 7117.614746 | 2.123328      | 0.261824 |
| 2 | SpiceJet  | 7290.845703 | 2.109421      | 0.308847 |

```
==== 최적 항공사 =====  
Air_India
```

**감사합니다.**